

Instituto Tecnológico de Costa Rica

Aseguramiento de Calidad de Software

Alicia Salazar Hernández

Laboratorio de pruebas de integración

Susana Feng

Ximena Molina

I Semestre

2026

Índice

Matriz de casos de prueba.....	2
Reporte de ejecución de Postman.....	5
Reporte de ejecución de Pytest.....	19
Reporte de ejecución de Newman.....	24
Reporte final de pruebas de integración.....	27
Tabla de métricas.....	27
Hallazgos y defectos.....	27
Comparación técnica: Postman vs Pytest.....	28
Conclusiones.....	28
¿Qué diferencia observó entre una prueba de API aislada y una prueba de integración?..	28
¿Qué defectos podrían pasar desapercibidos si solo se prueba la interfaz gráfica?.....	29
¿Qué validaciones fueron más importantes: estado HTTP, contrato JSON, reglas de negocio o persistencia? Justifique.....	29
¿Qué riesgos tendría integrar un proveedor externo de pagos sin pruebas automatizadas?.....	30
¿Qué mejoraría en su estrategia de pruebas si este proyecto fuera para producción?..	30
Referencias bibliográficas.....	31

Matriz de casos de prueba

A continuación se presenta la matriz de los casos de prueba planteados para la revisión del programa. Cabe resaltar que los estados HTTP utilizados fueron consultados de MDN Web Docs (2025).

ID	Caso	Endpoint/acción	Resultado esperado	Validación
TC-01	Listar todas las órdenes con paginación	GET/order/?skip=0&limit=10	Status 200 (OK), retorna un body con el total (número), skip (número), limit (número), data (array con número de elementos según limit). "Data" contiene objetos con _id (formato ObjectId), cliente_id (formato ObjectId), fecha (formato ISO 8601), canal (string), moneda (string), total (número mayor a 0) y items (array). "Items" contiene objetos con producto_id (formato ObjectId), cantidad (número mayor a 0) y precio_unit (número mayor a 0).	Estado HTTP + Contrato + Contenido + Formato
TC-02	Listar todas las órdenes con paginación inválida	GET/order/?skip=0&limit="10"	Status 422 (Unprocessable Content), retorna un body con un detail (array). El cual contiene un objeto con type (string indicando un error de tipo "int_parsing"), loc (array de strings de donde proviene el error), msg (string detallando que el error se debe a que no se puede parsear un string como un número entero) e input (string con la entrada inválida).	Estado HTTP + Contrato + Contenido + Formato
TC-03	Listar orden específica	GET/order/{id}	Status 200 (OK), retorna un body con un objeto que contiene _id (formato ObjectId), cliente_id (formato ObjectId), fecha (formato ISO 8601), canal (string), moneda (string), total (número mayor a 0) y items (array). "Items" contiene objetos con producto_id (formato ObjectId), cantidad (número mayor a 0) y precio_unit (número mayor a 0).	Estado HTTP + Contrato + Contenido + Formato
TC-04	Listar orden	GET/order/{id}	Status 404 (Not Found), retorna	Estado HTTP +

	específica con id no existente		un body con un detail (string) que indica que la orden con el {id} no pudo ser encontrado.	Contrato + Contenido
TC-05	Crear una orden	POST/order (Entradas se ingresan en el body)	Status 401 (Created), retorna un body con el order_id (formato ObjectId).	Estado HTTP + Contrato + Formato
TC-06	Crear una orden con entradas inválidas	POST/order (Entradas se ingresan en el body)	Status 422 (Unprocessable Content), retorna un body con un detail que puede ser un array o un string indicando que el cliente_id debe ser un ObjectId. Si el detail es un array, debe contener un objeto con type (string), loc (array), msg (string que describa el error), input y podría contener ctx (objeto JSON).	Estado HTTP + Contrato + Contenido + Formato
TC-07	Actualizar una orden	PUT/orders/{id}	Status 200, retorna un body con el campo message de tipo String. Este campo es igual a "order updated".	Estado HTTP + Contrato + Contenido
TC-08	Actualizar una orden con datos incorrectos	PUT/orders/{id}	Status 422, retorna un body que contiene "detail" de tipo String. Este campo indica cuál valor mandado está incorrecto.	Estado HTTP + Contrato + Contenido
TC-09	Eliminar una orden	DELETE/orders/{id}	Status 200, retorna un body con el campo message de tipo String. Este campo es igual a "order deleted".	Estado HTTP + Contrato + Contenido
TC-10	Eliminar orden con ID que no existe	DELETE/orders/{id}	Status 404, retorna un body con el campo detail de tipo String. Este campo indica que la orden no fue encontrada.	Estado HTTP + Contrato + Contenido
TC-11	Listar todos los clientes	Get/clients/	Status 200, retorna un body que contiene total (número), skip (número), limit (número), data (array). Data contiene objetos con _id, nombre, email, genero, pais, preferencias y creado. El atributo _id debe venir en formato ObjectId, el email con @ y el creado en formato ISO 8601.	Estado HTTP + Contrato + Contenido + Formato
TC-12	Listar	Get/clients/?skip=	Status 400, retorna un body con	Estado HTTP +

	clientes con parámetro de paginación inválido.	0&limit="Hola"	el campo detail de tipo String. Este campo indica formato de limit inválido	Contrato + Contenido + Formato
TC-13	Listar cliente específico	GET/clients/{id}	Status 200, retorna un body que contiene los campos _id de tipo ObjectId, nombre de tipo string, email de tipo string, genero de tipo string, pais de tipo string, preferencias de tipo object y creado de tipo string. El campo _id tiene formato ObjectId, email tiene formato válido con @ y creado tiene formato ISO 8601.	Estado HTTP + Contrato + Contenido
TC-14	Lista cliente específico que no existe	Get/clients/{id}	Status 404, retorna un body con el campo de detail (string). Este campo contiene el mensaje "Cliente x not found", donde la x corresponde al id del cliente ingresado en la petición.	Estado HTTP + Contrato + Contenido
TC-15	Listar todos los productos con paginación	GET/products/?skip=0&limit=10	Status 200 (OK), retorna un body con el total (número), skip (número), limit (número), data (array con número de elementos según limit). "Data" contiene objetos con _id (formato ObjectId), codigo_mongo (string), nombre (string), categoria (string) y equivalencias (objeto JSON). "Equivalencias" debe contener un objeto de codigo_alt (string) y opcionalmente puede tener otro objeto de sku (string).	Estado HTTP + Contrato + Contenido + Formato
TC-16	Listar todos los productos con paginación inválida	GET/products/?skip=0&limit="cinco"	Status 422 (Unprocessable Content), retorna un body con un detail (array). El cual contiene un objeto con type (string indicando un error de tipo "int_parsing"), loc (array de strings de donde proviene el error), msg (string detallando que el error se debe a que no se puede parsear un string como un número entero) e input (string con la entrada inválida).	Estado HTTP + Contrato + Contenido + Formato
TC-17	Listar	GET/products/{id}	Status 200 (OK), retorna un body	Estado HTTP +

	producto específico		con un objeto que contiene <code>_id</code> (formato ObjectId), <code>codigo_mongo</code> (string), <code>nombre</code> (string), <code>categoria</code> (string) y <code>equivalencias</code> (objeto JSON). “Equivalencias” debe contener un objeto de <code>codigo_alt</code> (string) y opcionalmente puede tener otro objeto de <code>sku</code> (string).	Contrato + Contenido + Formato
TC-18	Listar producto específico con id no existente	GET/products/{id}	Status 404 (Not Found), retorna un body con un detail (string) que indica que el producto con el {id} no pudo ser encontrado.	Estado HTTP + Contrato + Contenido

Reporte de ejecución de Postman

En esta sección se estarán presentando las pruebas de la realización de los distintos ejercicios realizados en Postman. Estos ejercicios corresponden a los casos de pruebas planteados en la matriz de la sección anterior.

Primero se creó la colección con la que se estará trabajando llamada “Integración API - Servicios”. Seguidamente se dividió en carpetas de acuerdo a las entidades que contiene el servicio API. En la siguiente figura se puede observar con mayor claridad la colección.

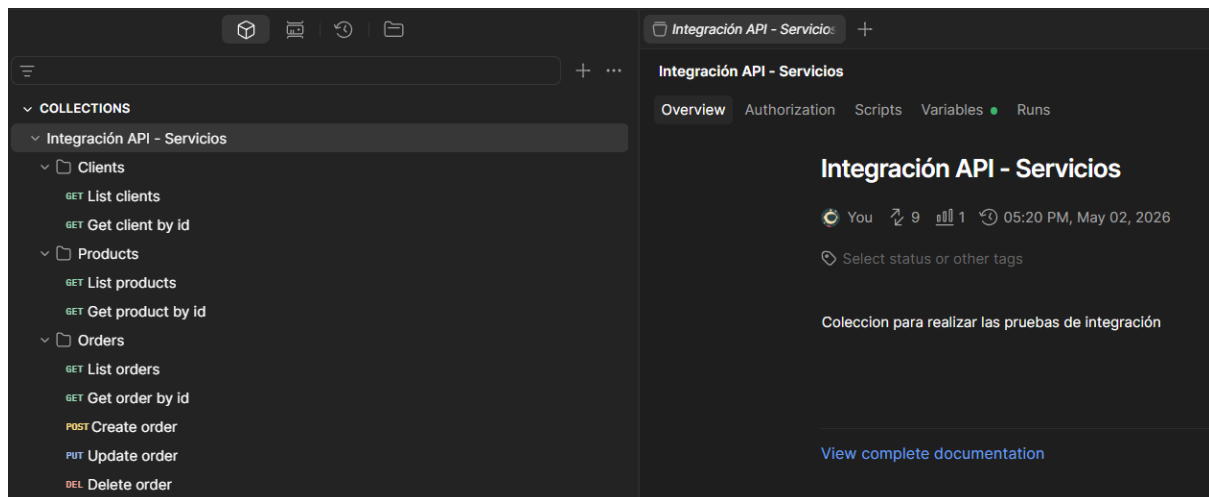


Figura 1. Captura de pantalla de postman mostrando la colección

Esta colección será probada en un ambiente de prueba específico, el cual corresponde al llamado QA_API_LAB. En este se definieron las variables necesarias para la llamada a los endpoints como la `base_url`, `product_id`, entre otros. En la siguiente captura de pantalla se puede observar el ambiente de prueba sin las credenciales reales utilizadas.

Variable	Value	Q	...
base_url	http://TU_URL_BASE		
product_id	PRODUCT_ID		
client_id	CLIENT_ID		
order_id	ORDER_ID		
Add variable			

Figura 2. Captura de pantalla de postman mostrando el ambiente de prueba

En las siguientes figuras se muestran las pruebas correspondientes a la acción GET de todas las órdenes. Estas corresponden a la TC-01: Listar todas las órdenes con paginación y TC-02: Listar todas las órdenes con paginación inválida.

Integración API - Servicios > Orders > TC-01: Listar todas las órdenes con paginación

GET {{baseUri}} /order/?skip=0&limit=10

Docs Params Authorization Headers 6 Body Scripts Settings

Pre-request

Post-response

```

1 // Estado HTTP
2 pm.test("Código de status es 200", function () {
3     pm.response.to.have.status(200);
4 });
5
6 // Contrato: validar claves principales
7 pm.test("Body de respuesta tiene los campos total, skip, limit y data.", function () {
8     let jsonData = pm.response.json();
9     pm.expect(jsonData).to.have.keys(["total", "skip", "limit", "data"]);
10    pm.expect(jsonData.data).to.be.an("array");
11 });
12
13 // Contenido: validar coherencia de valores
14 pm.test("Coherencia de valores, total y precio_unit son mayores a 0 y el número de items es igual al límite.", function () {
15     let jsonData = pm.response.json();
16     pm.expect(jsonData.total).to.be.above(0);
17     pm.expect(jsonData.data[0].items[0].precio_unit).to.be.above(0);
18     pm.expect(jsonData.data.length).to.equal(jsonData.limit);
19 });
20
21 // Formato: validar ObjectId y fecha ISO
22 pm.test("Los datos de los objetos del campo Data e Items tienen el formato correcto.", function () {
23     let jsonData = pm.response.json();
24     let regexObjectId = /^[0-9a-fa-f]{24}$/;
25     let regexISODate = /^d{4}-d{2}-d{2}Td{2}:d{2}:d{2}$/;
26 }

```

Body Cookies Headers 4 Test Results 4/4 200 OK · 162 ms · 3.87 KB

Filter Results

- PASSED Código de status es 200
- PASSED Body de respuesta tiene los campos total, skip, limit y data.
- PASSED Coherencia de valores, total y precio_unit son mayores a 0 y el número de items es igual al límite.
- PASSED Los datos de los objetos del campo Data e Items tienen el formato correcto.

Figura 3. Captura de pantalla de postman mostrando la prueba positiva (TC-01)

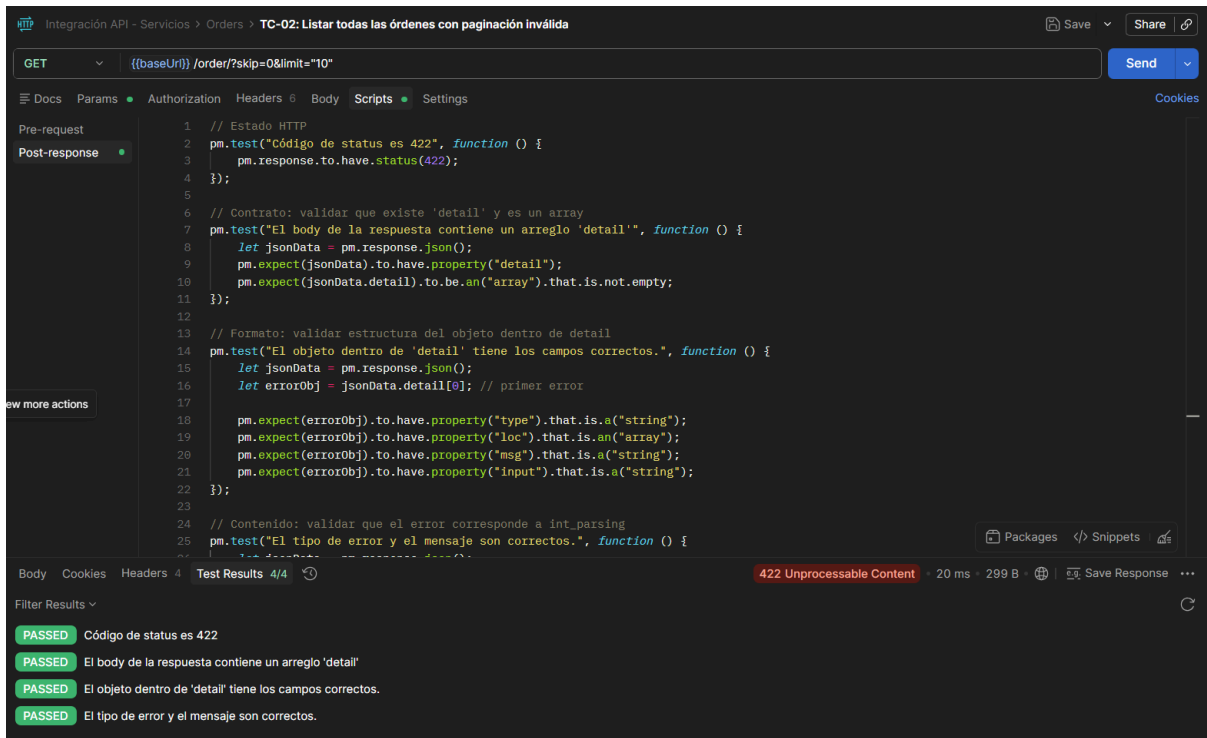


Figura 4. Captura de pantalla de postman mostrando la prueba negativa (TC-02)

En las siguientes figuras se muestran las pruebas correspondientes a la acción GET de una orden en específica. Estas corresponden a la TC-03: Listar orden específica con id y TC-04: Listar orden específica con id no existente.

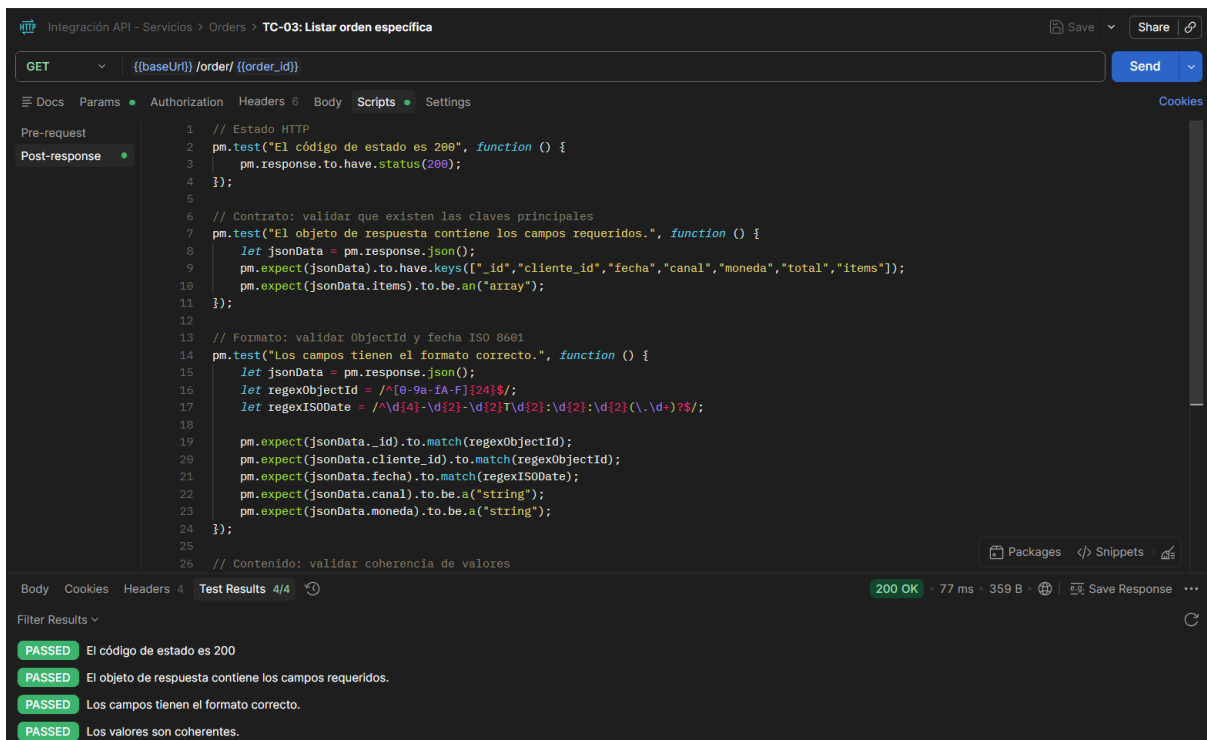


Figura 5. Captura de pantalla de postman mostrando la prueba positiva (TC-03)

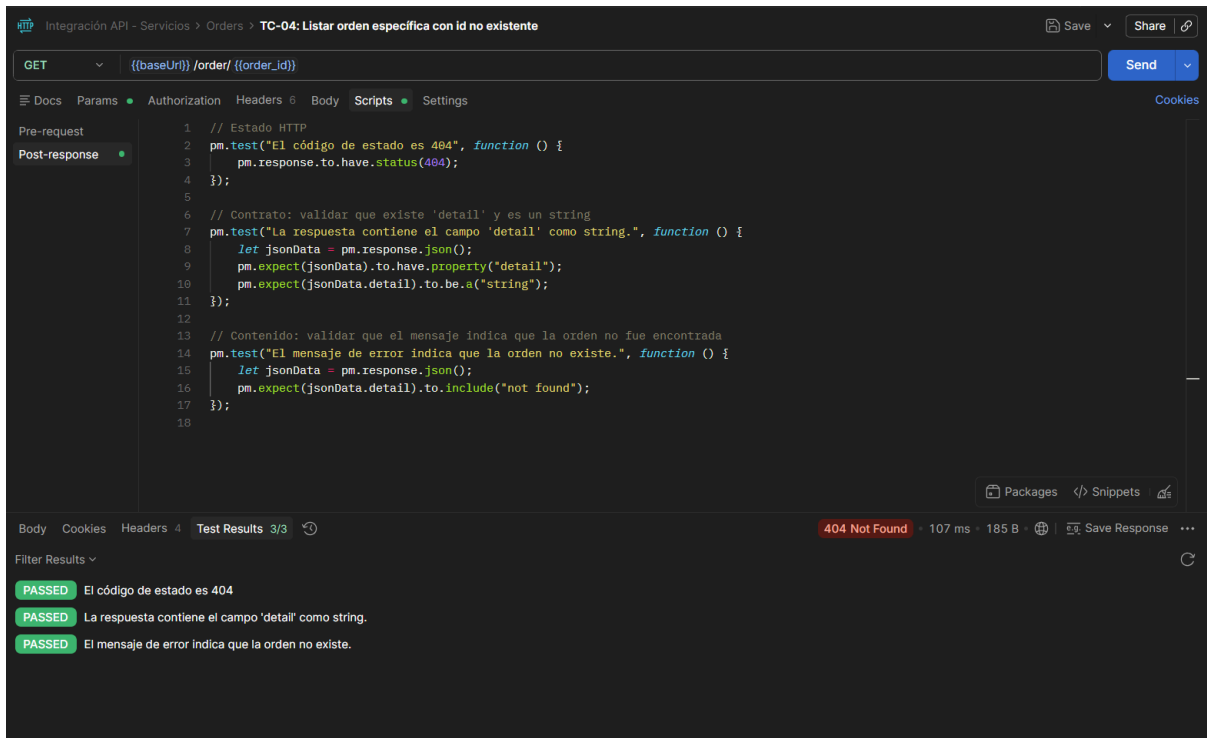


Figura 6. Captura de pantalla de postman mostrando la prueba negativa (TC-04)

En las siguientes figuras se muestran las pruebas correspondientes a la acción POST de una nueva orden. Estas corresponden a la TC-05: Crear una orden y TC-06: Crear una orden con entradas inválidas.

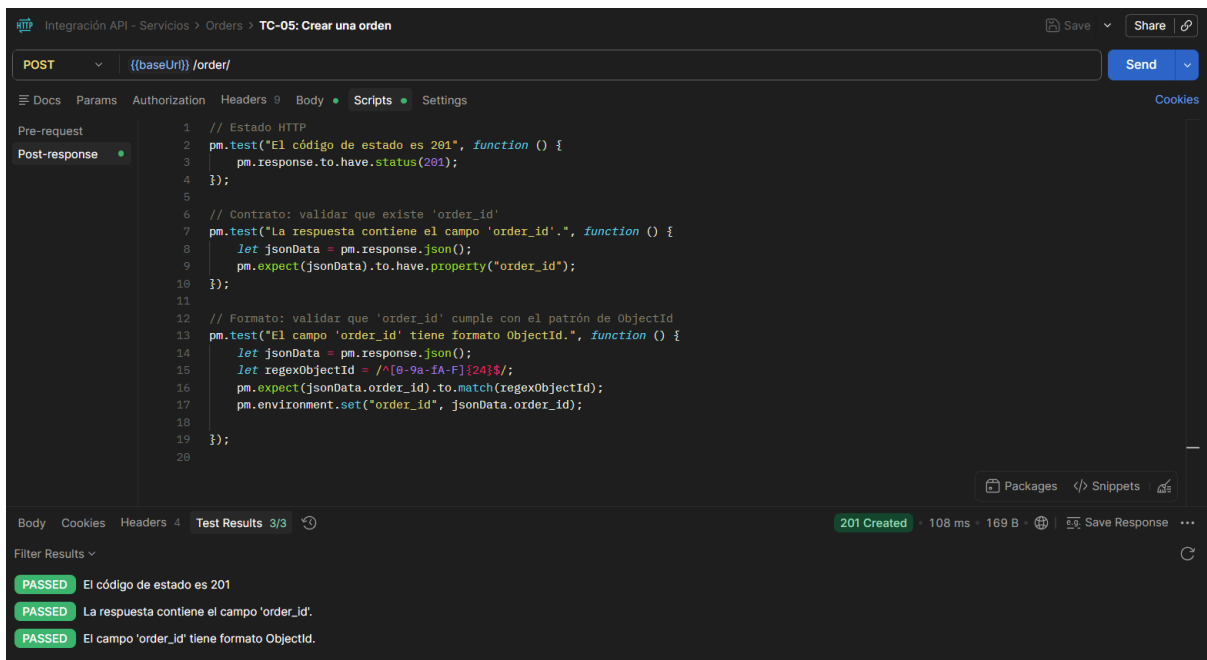


Figura 7. Captura de pantalla de postman mostrando la prueba positiva (TC-05)

Cabe destacar que en la prueba negativa (TC-06) hay diversos casos de datos de entrada inválidos y dependiendo del caso, la forma de la respuesta puede variar un poco.

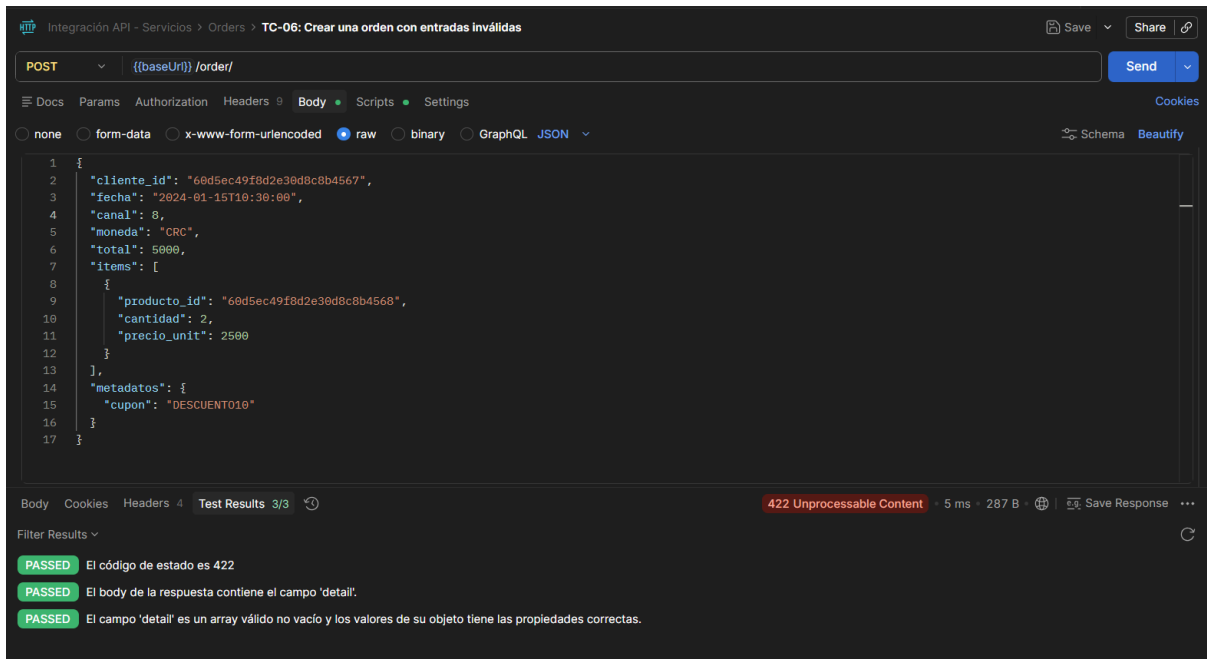


Figura 8. Captura de pantalla de postman mostrando la prueba negativa: enviar tipo de dato incorrecto (TC-06)

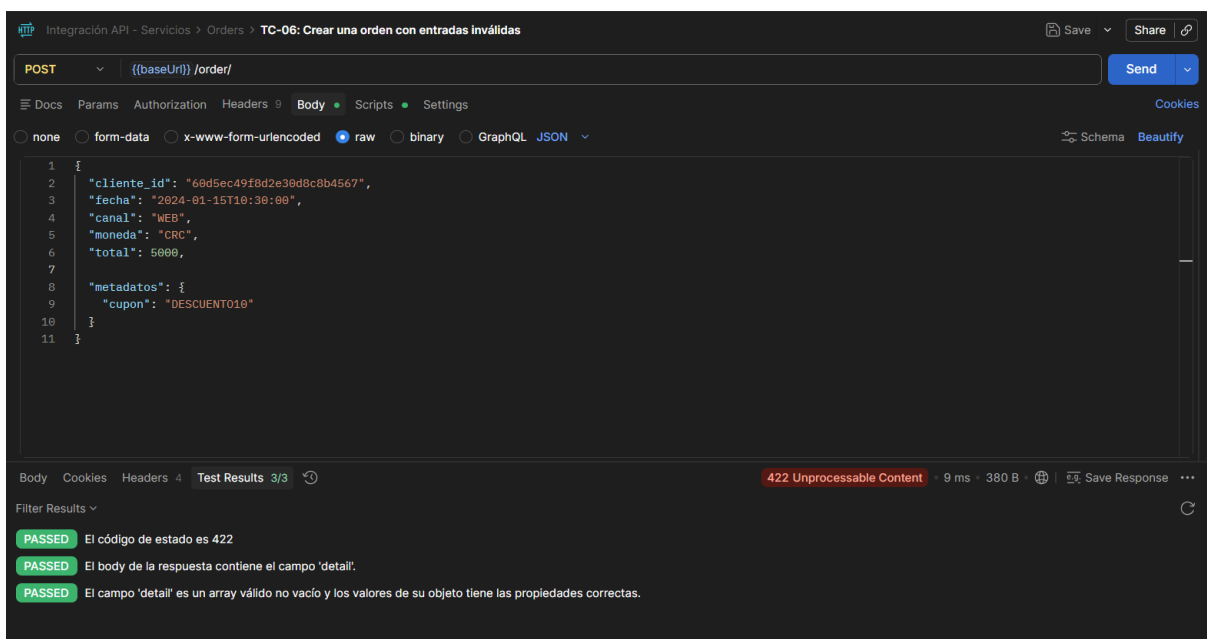


Figura 9. Captura de pantalla de postman mostrando la prueba negativa: body incompleto (TC-06)

Como se ve en la Figura 10, el sistema no valida que las cantidades ingresada sean mayor de cero y en vez de lanzar el código de error, el sistema permite cantidades menores a cero, por lo que falla todas las validaciones para este caso de uso.

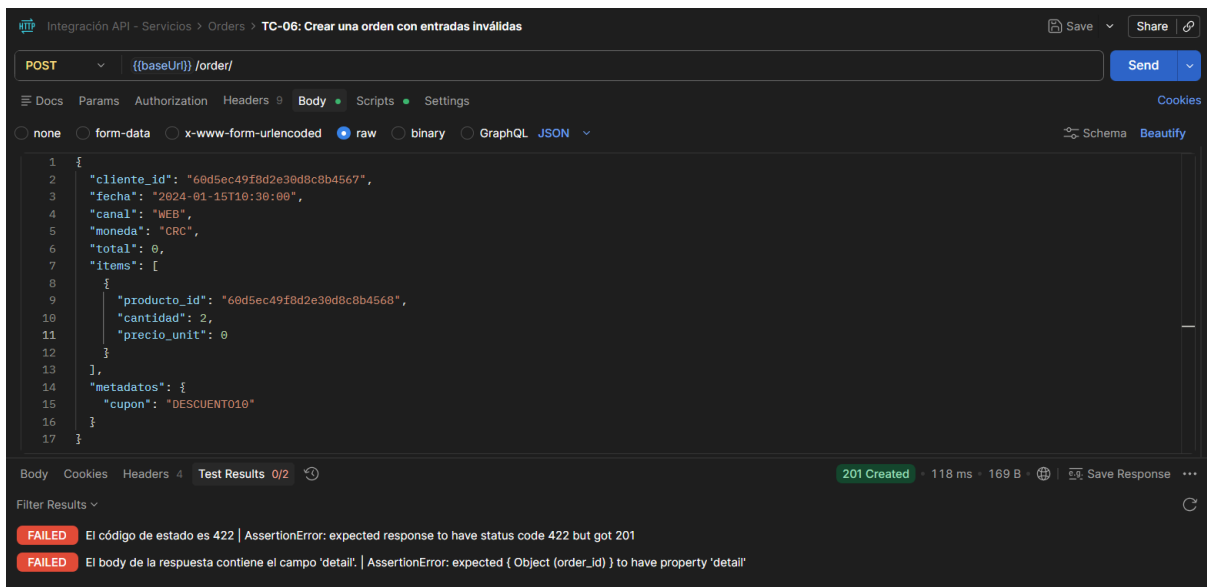


Figura 10. Captura de pantalla de postman mostrando la prueba negativa: enviar cantidad cero (TC-06)

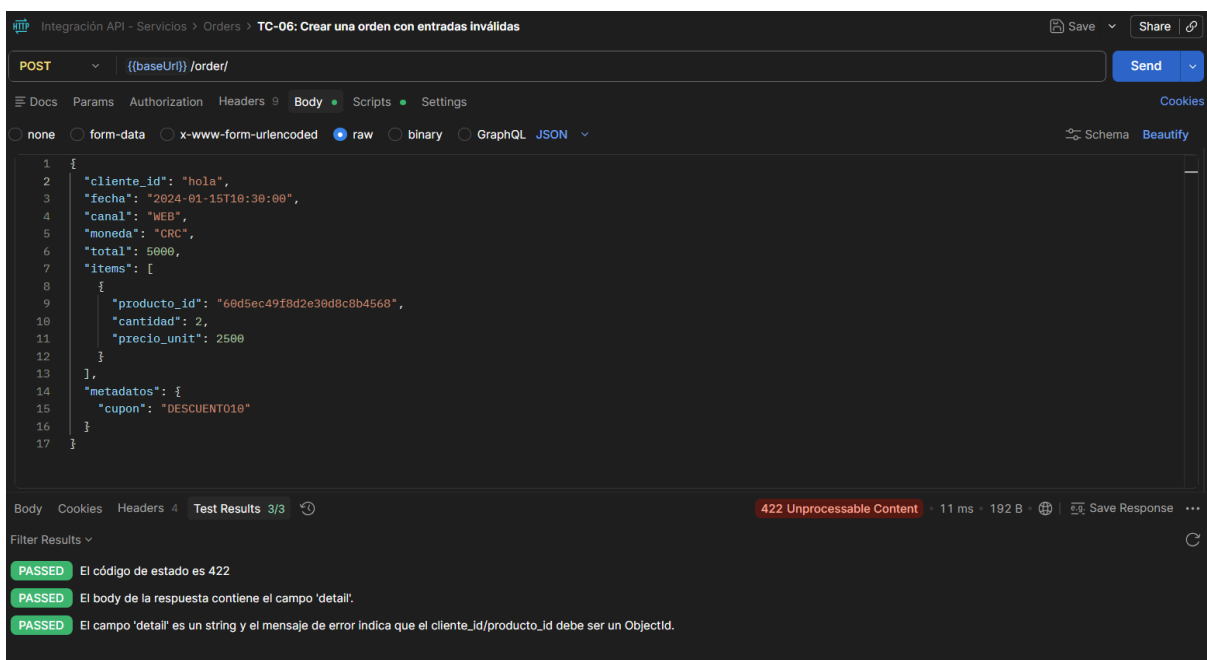


Figura 11. Captura de pantalla de postman mostrando la prueba negativa: enviar id en formato distinto al ObjectId (TC-06)

En las siguientes figuras se presentan las pruebas de Postman para TC-07: Actualizar una orden y TC-08: Actualizar una orden con datos incorrectos.

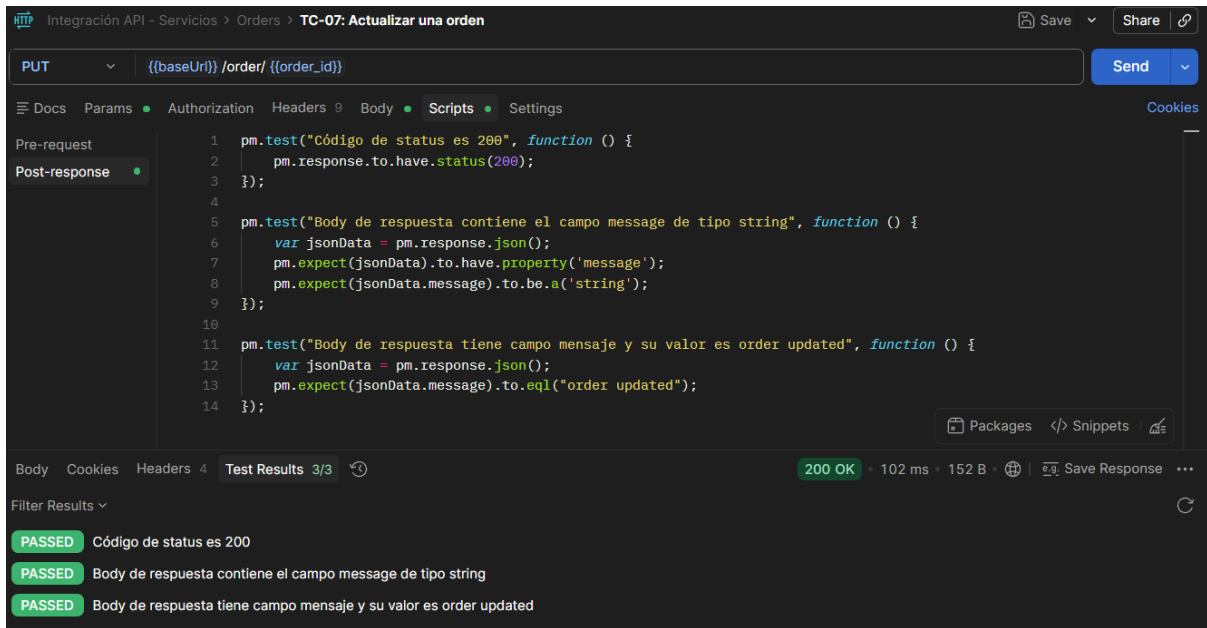


Figura 12. Captura de pantalla de postman mostrando la prueba positiva

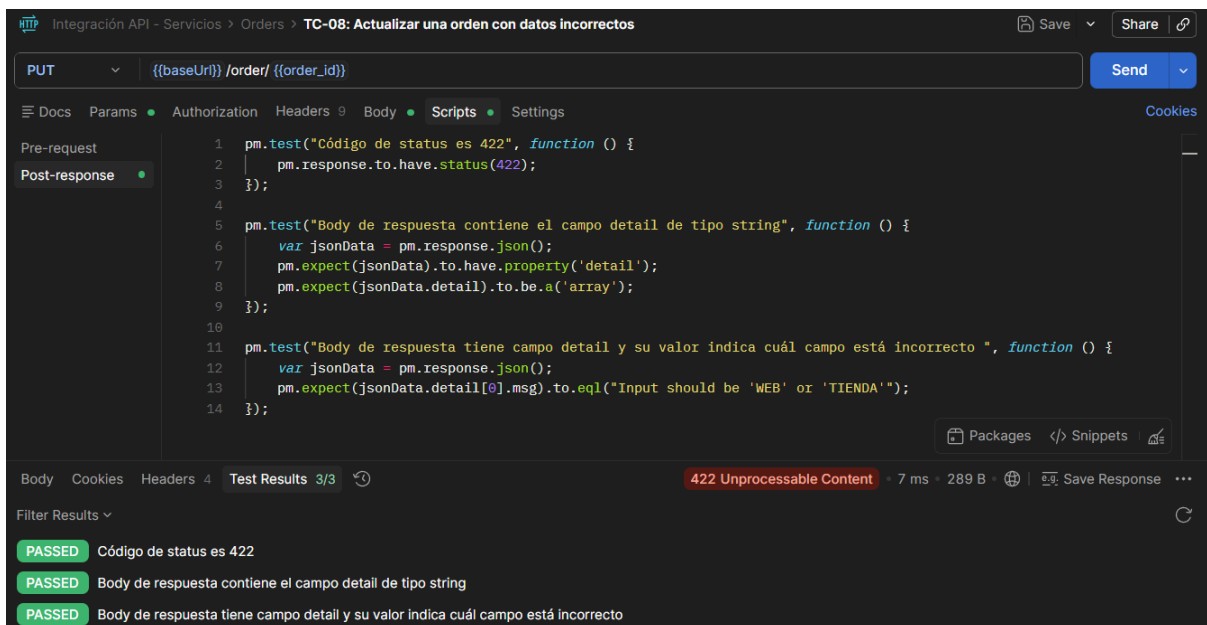


Figura 13. Captura de pantalla de postman mostrando la prueba negativa

En las siguientes figuras se muestran las pruebas correspondientes a la acción DELETE de órdenes. Estas corresponden a la TC-09: Eliminar una orden y TC-10: Eliminar una orden con Id.

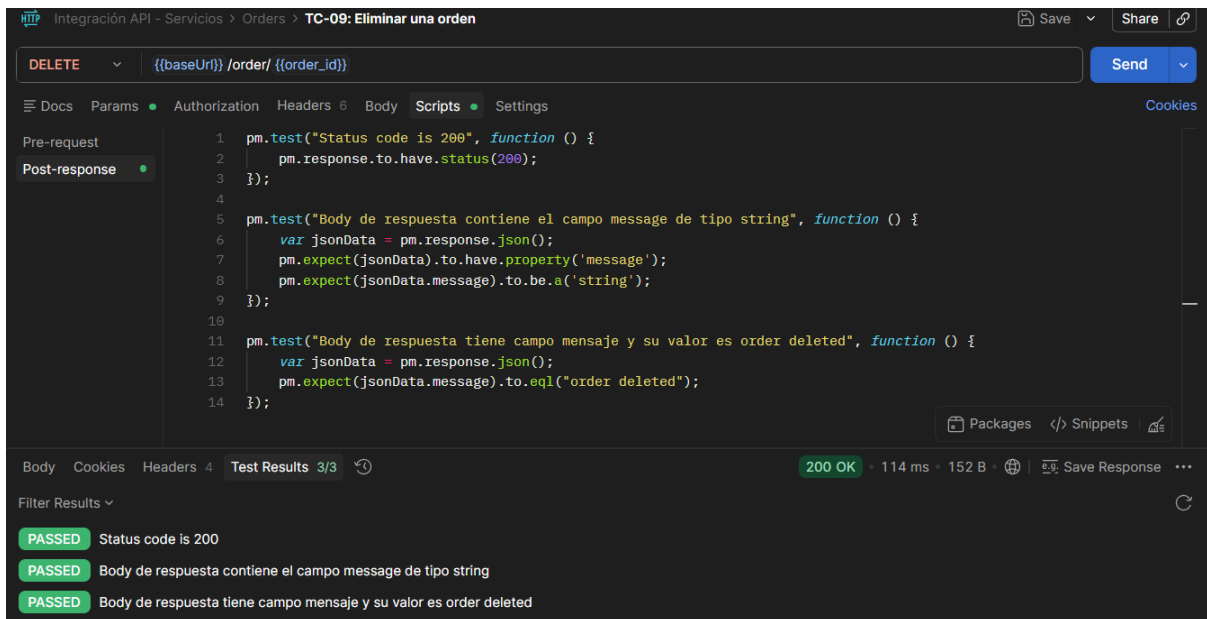


Figura 14. Captura de pantalla de postman mostrando la prueba positiva

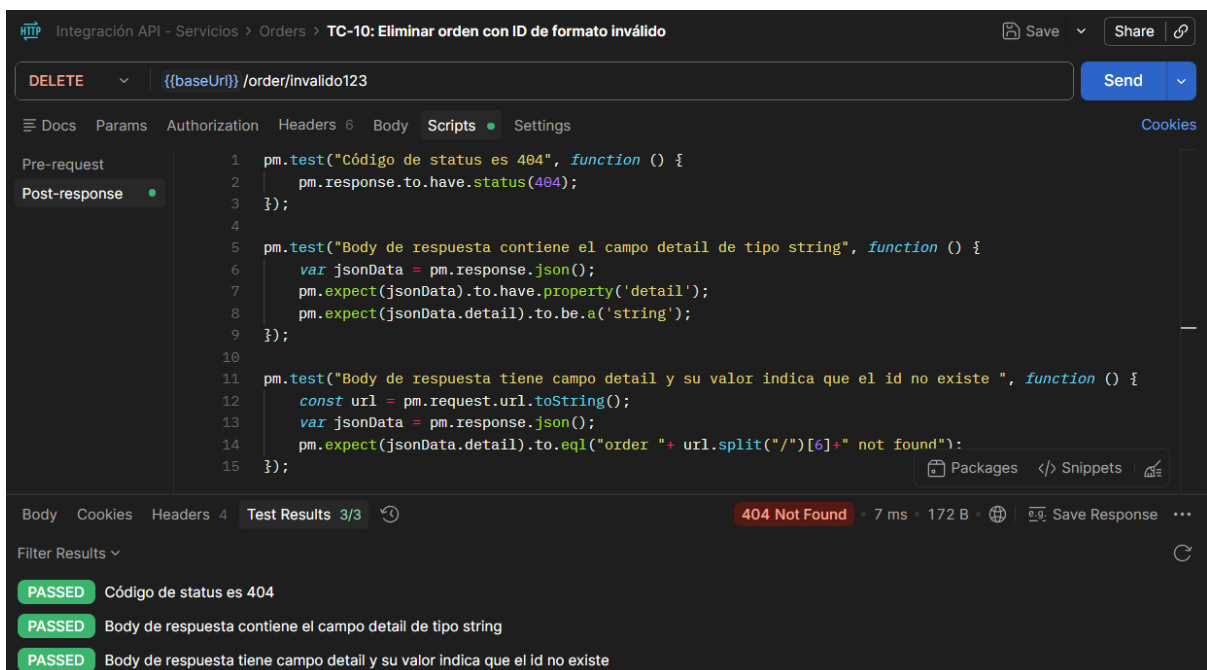


Figura 15. Captura de pantalla de postman mostrando la prueba negativo

En las siguientes figuras se muestran las pruebas realizadas para el GET de clientes en postman. Se presenta tanto una prueba positiva como una negativa. Cabe resaltar que la prueba negativa es fallida pues el endpoint no retorna un error, como debería hacerlo.

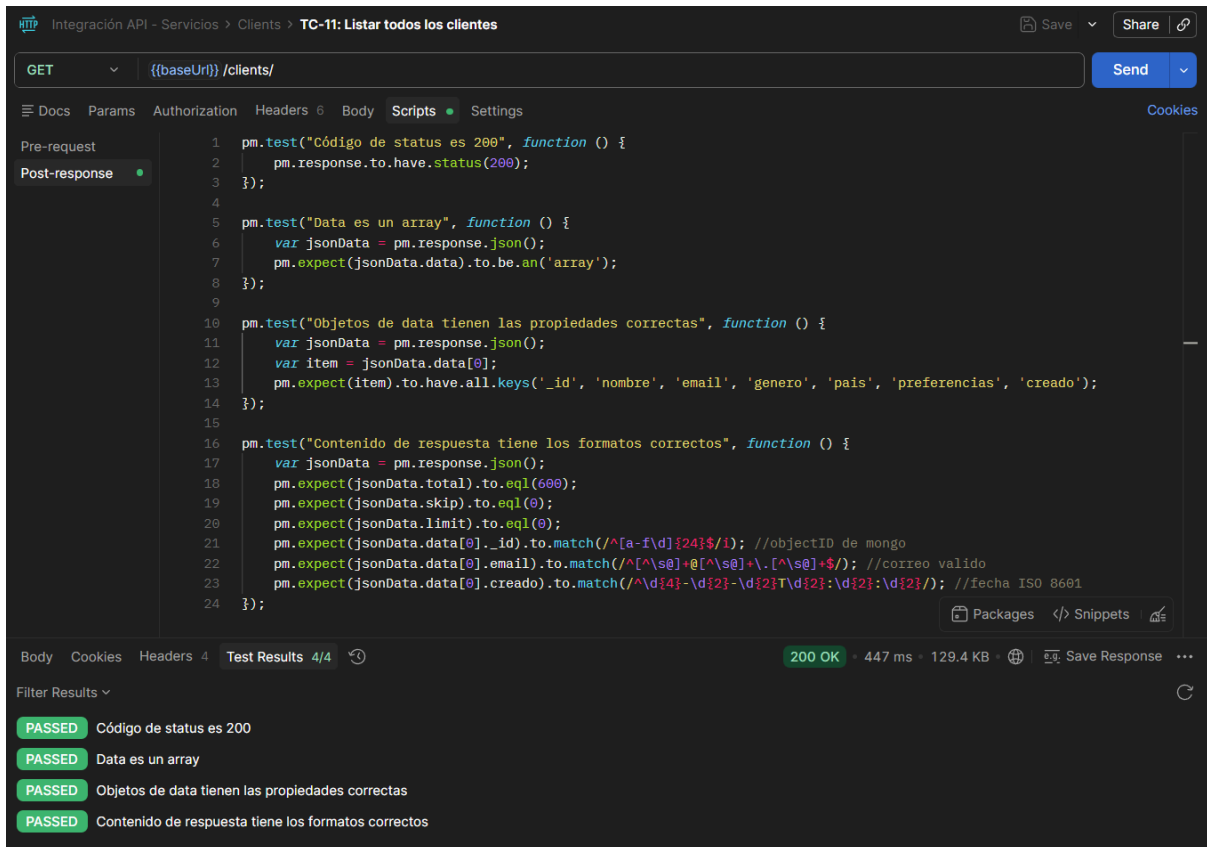


Figura 16. Captura de pantalla de postman mostrando la prueba positiva

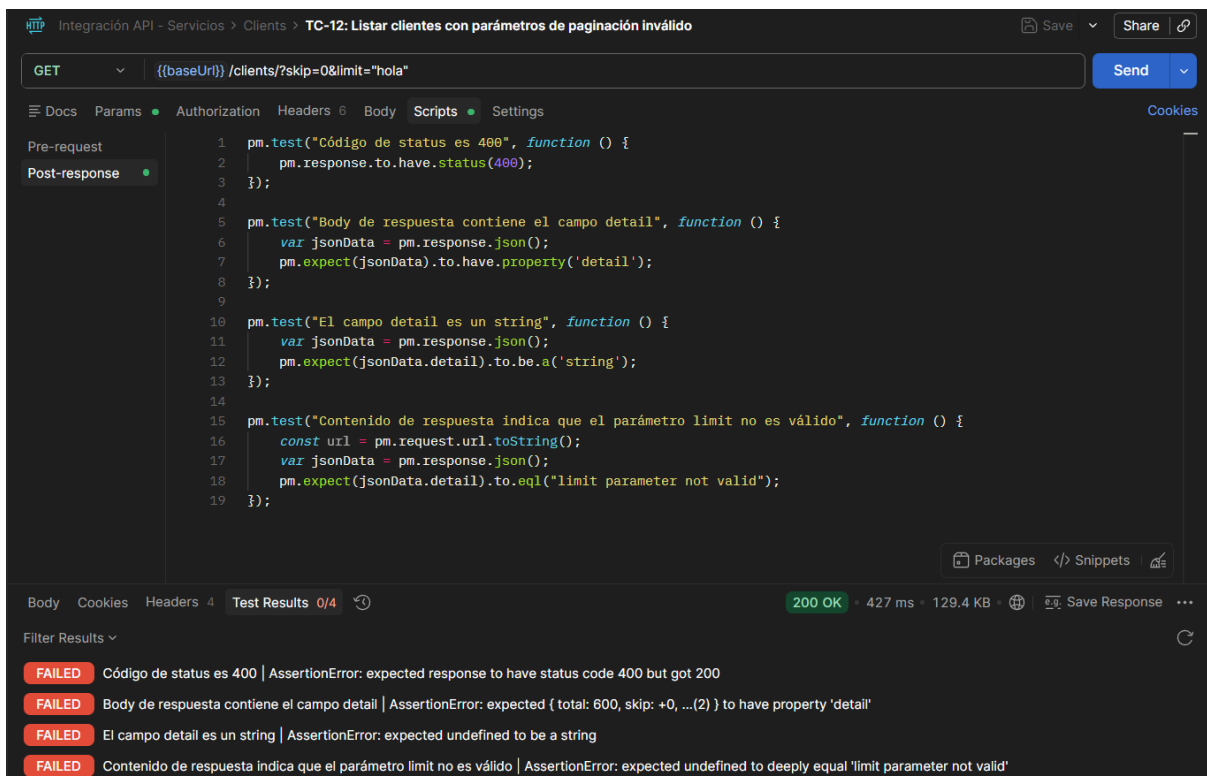


Figura 17. Captura de pantalla de postman mostrando la prueba negativa fallida

Las siguientes figuras corresponden a los get específicos de los clientes por medio de sus id. Estos son el TC-13: Listar cliente específico y el TC-14: Listar cliente específico que no existe respectivamente.

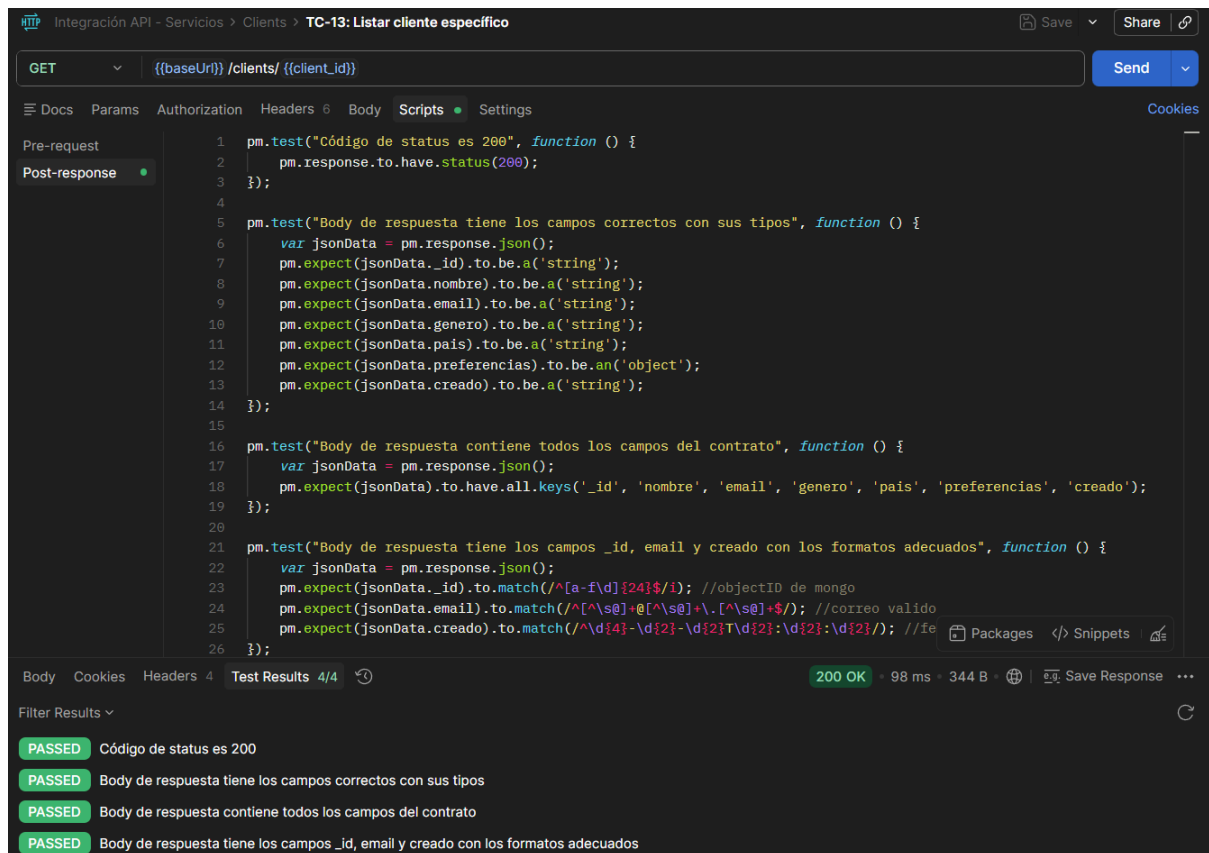


Figura 18. Captura de pantalla de postman mostrando la prueba positiva

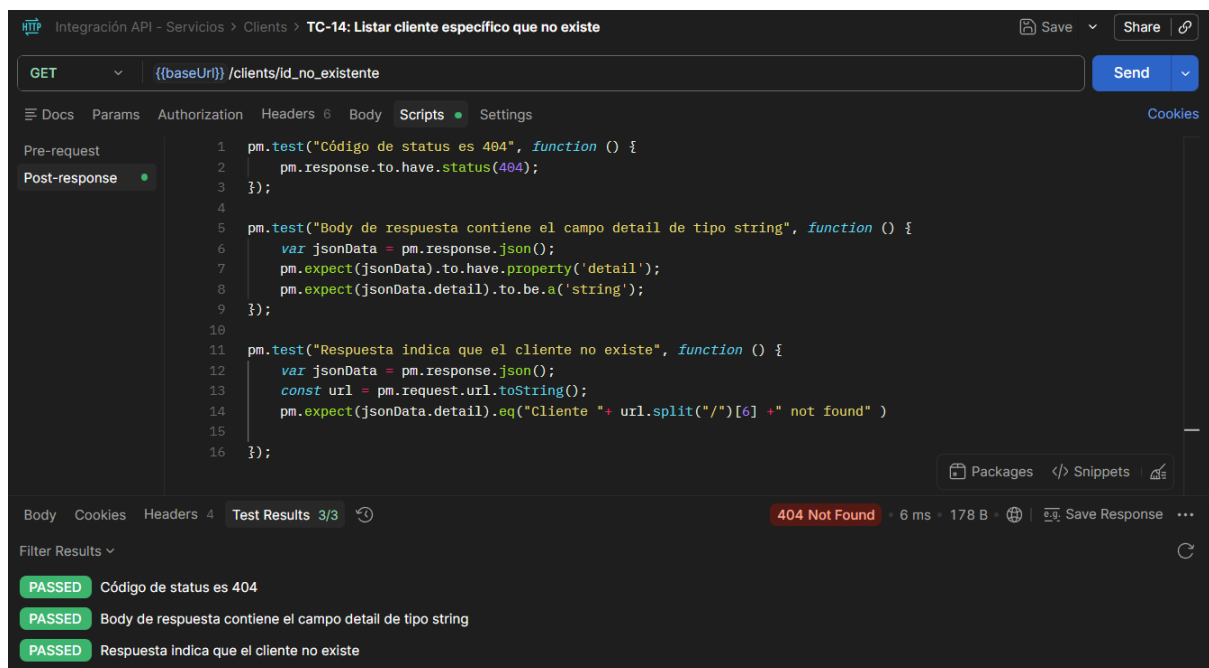


Figura 19. Captura de pantalla de postman mostrando la prueba negativa

En las siguientes figuras se muestran las pruebas correspondientes a la acción GET de todos los productos del catálogo. Estas corresponden a la TC-15: Listar todos los productos con paginación y TC-16: Listar todos los productos con paginación inválida.

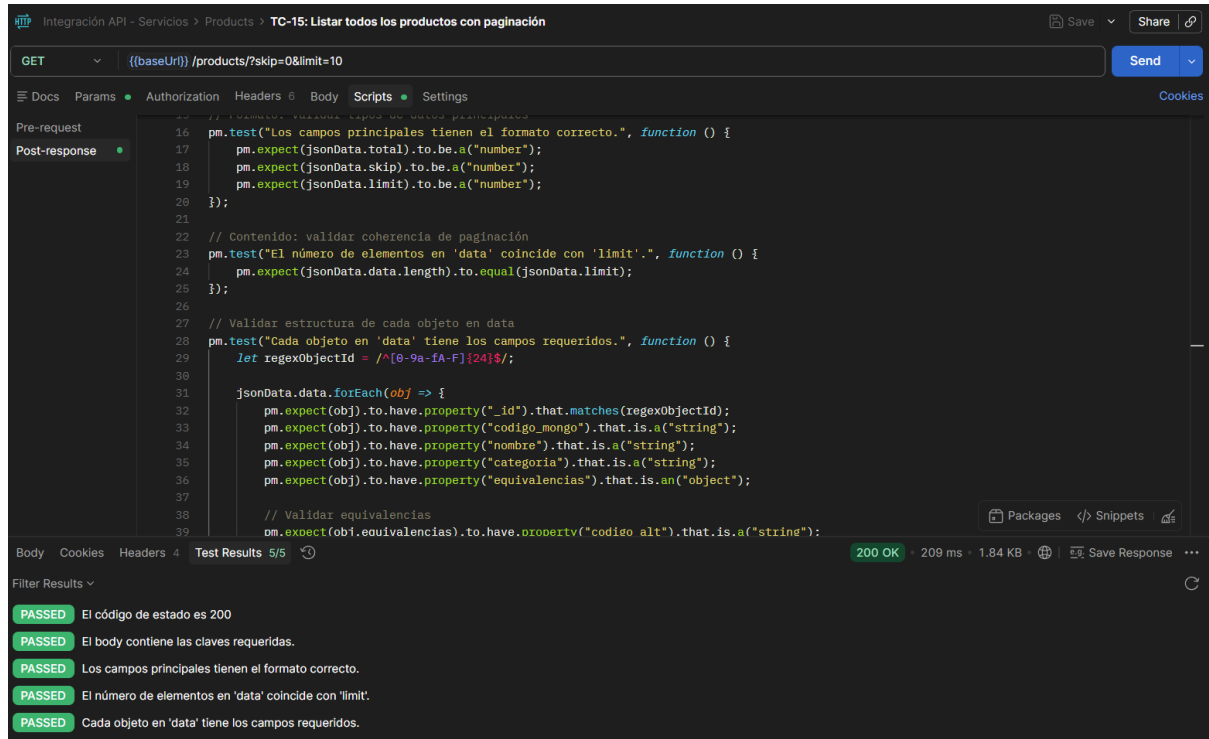


Figura 20. Captura de pantalla de postman mostrando la prueba positiva (TC-15)

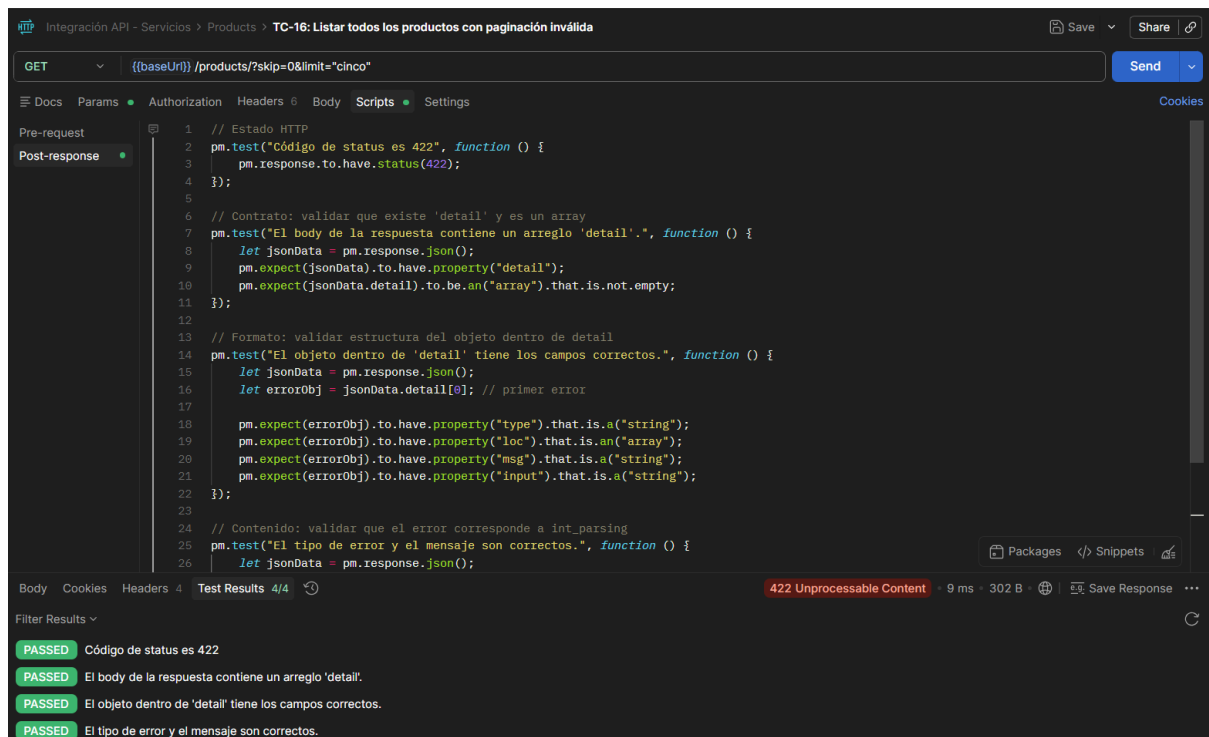


Figura 21. Captura de pantalla de postman mostrando la prueba negativa (TC-16)

En las siguientes figuras se muestran las pruebas correspondientes a la acción GET de un producto en específico del catálogo. Estas corresponden a la TC-17: Listar producto específico con id y TC-18: Listar producto específico con id no existente.

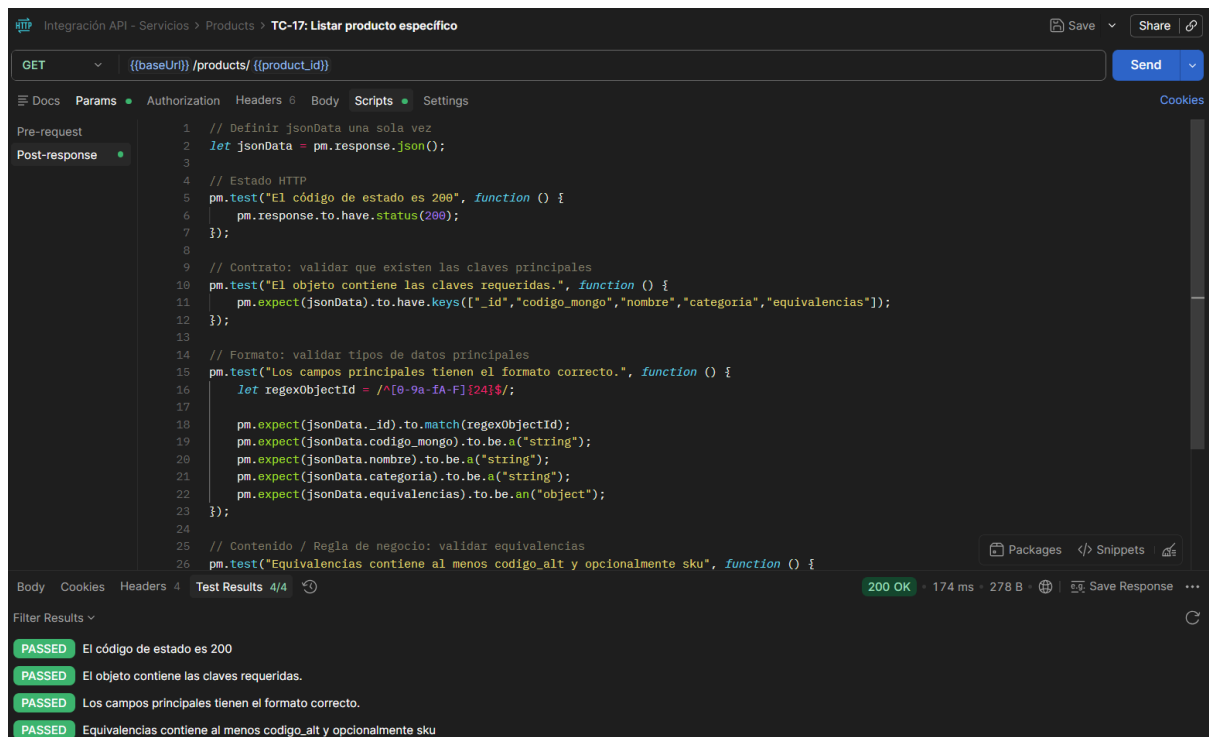


Figura 22. Captura de pantalla de postman mostrando la prueba positiva (TC-17)

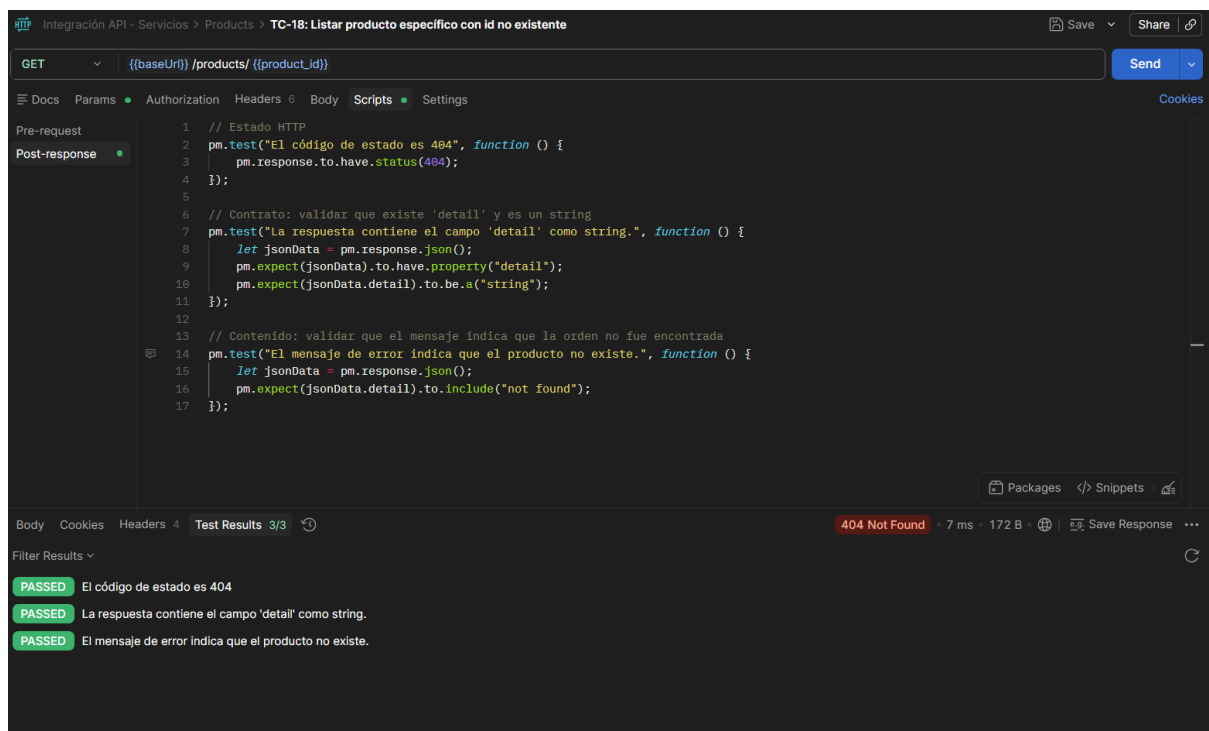


Figura 23. Captura de pantalla de postman mostrando la prueba negativa (TC-18)

Seguidamente, se realizo un total de tres iteraciones de la ejecución de la colección con el Colection Runner. En la siguiente figura se pueden observar todos los endpoints que fueron incluidos en el run.

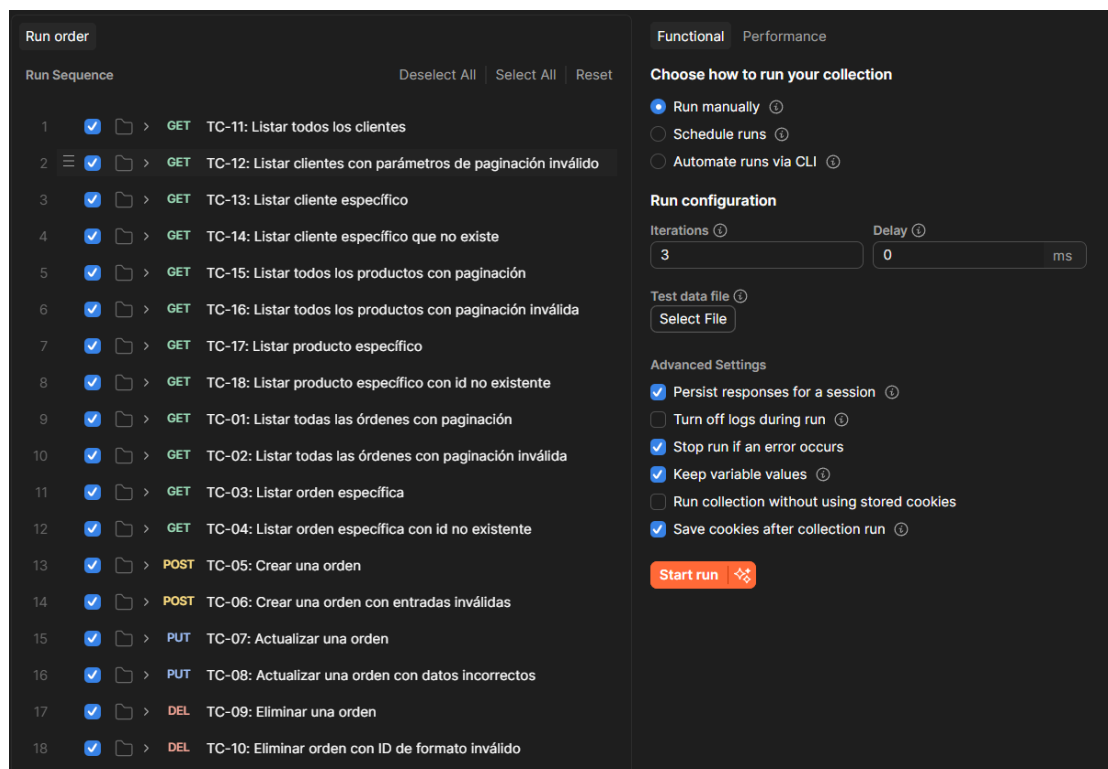


Figura 24. Captura de pantalla de postman mostrando el colection runner

En la siguiente captura se muestra el resumen de los resultados de las tres iteraciones realizadas.

Integración API - Servicios - Run results

Run today at 03:05:33 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	QA_APL_LAB	3	8s 858ms	192	0	73 ms

RUN SUMMARY

		1	2	3
> GET	TC-11: Listar todos los clientes	12	0	
> GET	TC-12: Listar clientes con parámetros de paginación inválido	0	12	X X X
> GET	TC-13: Listar cliente específico	12	0	
> GET	TC-14: Listar cliente específico que no existe	9	0	
> GET	TC-15: Listar todos los productos con paginación	15	0	
> GET	TC-16: Listar todos los productos con paginación inválida	12	0	
> GET	TC-17: Listar producto específico	12	0	
> GET	TC-18: Listar producto específico con id no existente	9	0	
> GET	TC-01: Listar todas las órdenes con paginación	12	0	
> GET	TC-02: Listar todas las órdenes con paginación inválida	12	0	
> GET	TC-04: Listar orden específica con id no existente	9	0	
> POST	TC-05: Crear una orden	9	0	
> GET	TC-03: Listar orden específica	12	0	
> POST	TC-06: Crear una orden con entradas inválidas	9	0	
> PUT	TC-07: Actualizar una orden	9	0	
> PUT	TC-08: Actualizar una orden con datos incorrectos	9	0	
> DELETE	TC-09: Eliminar una orden	9	0	
> DELETE	TC-10: Eliminar orden con ID de formato inválido	9	0	

Figura 25. Captura de pantalla de Postman mostrando resultados de corrida

Finalmente, se creó un mock server en Postman para simular un fallo en el servidor externo. En la siguiente captura de pantalla se muestra la configuración del servidor.

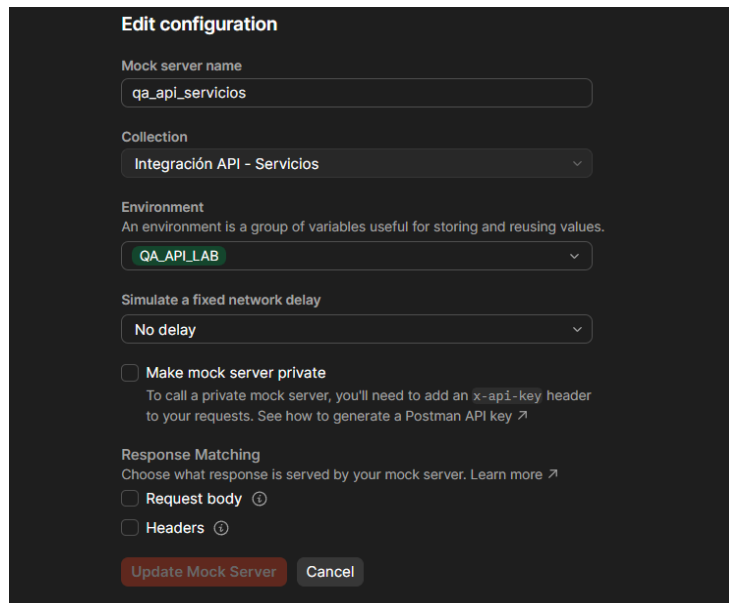


Figura 26. Captura de pantalla de la configuración del mock server en Postman

Luego se crea un ejemplo de una de las pruebas en Postman para que esta pueda ser utilizada para el mock server. En este caso, el ejemplo es de un servidor caído. Una vez puesto eso, se utiliza la URL del servidor para poder observar que devuelve el servidor. Se pueden observar todos los endpoints que se han ejecutado del servidor en el menú del mismo. A continuación se muestra una captura de distintas peticiones al servidor, para este caso, se hizo que todas los endpoints del mock de la entidad client devuelvan el error del servidor caído.

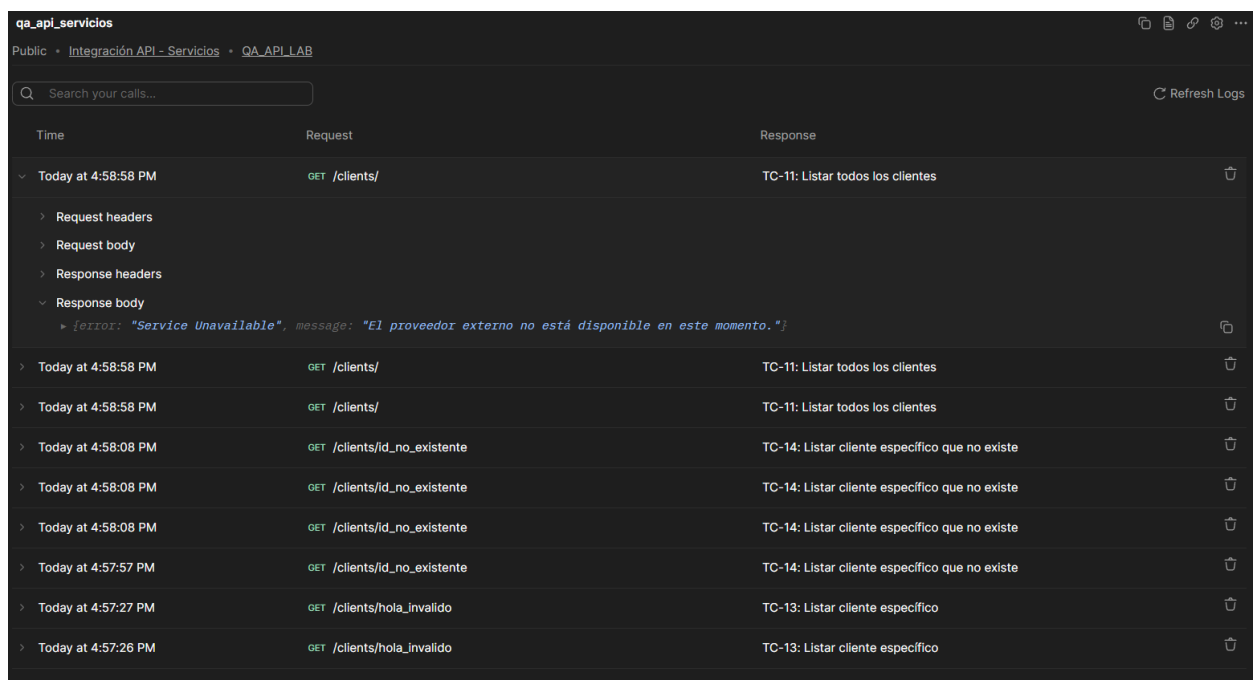


Figura 27. Captura de pantalla donde se observan las peticiones al mock server en Postman, y el mensaje que manda el servidor.

Reporte de ejecución de Pytest

Se creó una carpeta tests dedicada para las pruebas realizada con Pytest y SQLite en memoria, tal como se observa en la Figura 24. En el archivo “[conftest.py](#)” es donde se encuentra el código para inicializar la base de datos local temporal en memoria con SQLite. Está configurado para que los datos persistan por cada sesión de Pytest.

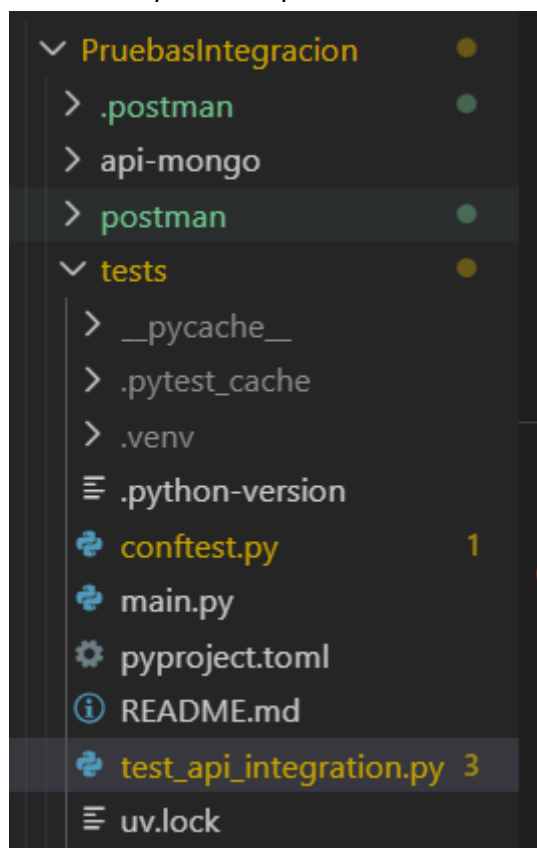


Figura 28. Captura de pantalla de la carpeta “tests” para la pruebas de integración con Pytest y SQLite

La siguiente figura es la prueba con Pytest para probar la consulta HTTP a la API de productos. Donde se valida el status de la respuesta, el contenido y formato de los elementos de productos consultados.

```
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\User\Desktop\QA\PruebasIntegracion\tests
configfile: pyproject.toml
collected 11 items

test_api_integration.py
  ● Test de consulta HTTP a la API de productos
  ✓ El API respondió con 200 OK
  ✓ La respuesta contiene la clave 'data'
  ✓ 'data' es una lista
  ✓ La lista 'data' contiene elementos
  ● Validando producto con _id: 692a992f056ff25d36115aab
  ✓ '_id' existe y es string
  ✓ 'codigo_mongo' existe y es string
  ✓ 'nombre' existe y es string
  ✓ 'categoria' existe y es string
  ✓ 'equivalencias' existe y es dict
  ✓ 'codigo_alt' existe y es string
  ✓ 'sku' existe y es string
  🎉 Todas las validaciones de producto pasaron correctamente
-----
```

Figura 29. Captura de pantalla de la prueba exitosa con Pytest para consulta HTTP a la API de productos

Luego, esta figura es la prueba para insertar un producto para crear una nueva orden dentro de la base de datos local temporal creada en SQLite. Se hizo la validación de que las API's a productos y a clientes funcionaran. Se validó que los datos consultados se insertaran en las tablas correspondientes antes de insertar la nueva orden. En la inserción de la nueva orden se validó que el nuevo registro haya sido insertado con todos los datos principales necesarios.

```
● Insertar un producto para crear una orden
  ✓ API productos respondió con 200 OK
  ✓ API productos devolvió lista con datos
  ✓ Producto consultado: 692a992f056ff25d36115aab - Cereal de avena 250 g
  ✓ API clientes respondió con 200 OK
  ✓ API clientes devolvió lista con datos
  ✓ Cliente consultado: 692a992f056ff25d36115853 - Vilma Jove Diéguez
  ✓ Producto insertado en tabla productos
  ✓ Cliente insertado en tabla clientes
  ✓ Canales de preferencias insertados en tabla clientes_canales
  ✓ Orden insertada en tabla ordenes
  ✓ Item insertado en tabla orden_items
  ✓ Validación producto insertado correctamente
  ✓ Validación cliente insertado correctamente
  ✓ Validación canales insertados correctamente
  ✓ Validación orden insertada correctamente
  ✓ Validación item insertado correctamente
  🎉 Todas las validaciones pasaron correctamente
-----
```

Figura 30. Captura de pantalla de la prueba exitosa con Pytest para prueba de inserción de producto para crear orden

Luego de validar la inserción de la nueva orden, se realizó una prueba para verificar que las cantidades insertadas de producto, el precio unitario y el monto total hayan sido insertados correctamente.

```

• Validar que el total de cantidades coincida con lo insertado
✓ Se encontró una orden en la tabla ordenes
• Orden seleccionada: 69f8322d5235c9ea31f86f7c con total guardado 7500
✓ Se encontró item asociado a la orden 69f8322d5235c9ea31f86f7c
• Item recuperado: cantidad=5, precio_unit=1500
✓ La cantidad coincide con la esperada
✓ El precio_unit coincide con el esperado
✓ El total guardado coincide con la suma cantidad x precio_unit
🎉 Todas las validaciones de la orden pasaron correctamente
-----

```

Figura 31. Captura de pantalla de la prueba exitosa con Pytest para verificar cantidades insertadas correctamente en la orden

Seguidamente, se realizó una validación de las reglas de negocio aplicadas sobre los valores de “cantidad”, “precio_unit” y “total”, las cuales deben ser números positivos mayores a 0. También se hizo una validación sobre “producto_id”, para verificar que el producto ingresado sí exista y esté registrado dentro de la base de datos. Además, que se valida que si se inserta una orden, esta no puede venir con su contenido vacío.

```

• Validar cantidades de acuerdo con las reglas del negocio
✓ Se encontraron 1 órdenes en la tabla

• Validando orden 69f8322d5235c9ea31f86f7c con total guardado 7500
✓ La orden 69f8322d5235c9ea31f86f7c tiene 1 items asociados
• Validando item con producto 692a992f056ff25d36115aab, cantidad=5, precio_unit=1500
✓ Cantidad positiva
✓ Precio positivo
✓ Producto existe en la tabla productos
✓ Total consistente en orden 69f8322d5235c9ea31f86f7c: guardado=7500, calculado=7500

🎉 Todas las validaciones de reglas de negocio pasaron correctamente
-----

```

Figura 32. Captura de pantalla de la prueba exitosa con Pytest para verificar reglas de negocio

En la siguiente figura se muestra la prueba del ingreso de un producto que no existe dentro de la base de datos. Se comprueba que si no existe, la orden o el item no debe poder ser insertado en la tabla para evitar tener registros erróneos.

```

• Validar el ingreso de un producto que no existe
🔥 Intentando insertar item con producto inexistente: noexiste1234567890abcd
✓ Se lanzó IntegrityError como se esperaba (producto inexistente no permitido)
• Items encontrados para la orden 69f8322d5235c9ea31f86f7c y producto noexiste1234567890abcd: []
✓ Validación final: no se insertaron items inválidos
🎉 Todas las validaciones de producto inexistente pasaron correctamente
-----

```

Figura 33. Captura de pantalla de la prueba exitosa con Pytest para validar el ingreso de un producto inexistente

Las siguientes figuras corresponden a la prueba de tiempo de respuesta de la API productos. Como el tiempo óptimo de respuesta debe ser de menos de un segundo, la prueba con la API fue fallida. En la Figura 31 se puede observar el detalle de que la prueba falló debido a que el tiempo de respuesta fue muy largo (>1 segundo).

```
Tiempo de respuesta del API: http://localhost:8000/products
❌Tiempo de respuesta: 2.626 segundos
✅ Status code 200 OK
F-----
```

Figura 34. Captura de pantalla de la prueba fallida con Pytest para probar el tiempo de respuesta de la API productos

```
===== FAILURES =====
_____ test_tiempo_respuesta_api _____

def test_tiempo_respuesta_api():
    print("-----")
    print(f"\nTiempo de respuesta del API: {BASE_URL}/products")

    inicio = time.perf_counter()
    response = requests.get(f"{BASE_URL}/products")
    fin = time.perf_counter()

    duracion = fin - inicio
    print(f"❌Tiempo de respuesta: {duracion:.3f} segundos")

    # Validaciones
    assert response.status_code == 200, "❌ El API no respondió con 200 OK"
    print("✅ Status code 200 OK")

    # Ejemplo: validar que el tiempo sea menor a 1 segundo
    > assert duracion < 1.0, f"❌ El API tardó demasiado: {duracion:.3f} segundos, límite (<1s)"
E   AssertionError: ❌ El API tardó demasiado: 2.626 segundos, límite (<1s)
E   assert 2.6264492999907816 < 1.0

test_api_integration.py:356: AssertionError
===== short test summary info =====
FAILED test_api_integration.py::test_tiempo_respuesta_api - AssertionError: ❌ El API tardó demasiado: 2.626 segundos, límite (<1s)
===== 1 failed, 10 passed in 10.26s =====
```

Figura 35. Captura de pantalla con el detalle de la prueba fallida con Pytest para probar el tiempo de respuesta de la API productos

En la siguiente figura se muestra las pruebas de contrato estricta para todas las tablas utilizadas dentro de la base de datos local temporal creada en SQLite. Se validó que todos los atributos de las tablas fueran del tipo de dato establecido para cada una.

```
Pruebas de contrato estricta: clientes
❌ Validando cliente 692a992f056ff25d36115853
✅ Cliente 692a992f056ff25d36115853 cumple contrato
.

Pruebas de contrato estricta: clientes_canales
❌ Validando canal TIENDA del cliente 692a992f056ff25d36115853
✅ Canal TIENDA del cliente 692a992f056ff25d36115853 cumple contrato
❌ Validando canal WEB del cliente 692a992f056ff25d36115853
✅ Canal WEB del cliente 692a992f056ff25d36115853 cumple contrato
.

Pruebas de contrato estricta: productos
❌ Validando producto 692a992f056ff25d36115aab
✅ Producto 692a992f056ff25d36115aab cumple contrato
.

Pruebas de contrato estricta: ordenes
❌ Validando orden 69f8322d5235c9ea31f86f7c
✅ Orden 69f8322d5235c9ea31f86f7c cumple contrato
.

Pruebas de contrato estricta: orden_items
❌ Validando item 1 de orden 69f8322d5235c9ea31f86f7c
✅ Item 1 de orden 69f8322d5235c9ea31f86f7c cumple contrato
.
```

Figura 36. Captura de pantalla de la prueba exitosa con Pytest para pruebas de contrato estricta para las tablas de la BD

En la siguiente figura se muestra la prueba de idempotencia que se realizó para verificar que cuando se cree una orden cada orden se mantenga única y no permite hacer órdenes con IDs idénticos.

```
• Validar idempotencia: crear múltiples órdenes con los mismos datos de negocio
✓ Primera orden creada: 69fa85e3f18b7f74c38d6a0a
✓ Segunda orden creada: 69fa85e3f18b7f74c38d6a0b
✓ Las órdenes tienen IDs únicos – no hay colisión
✓ Ambas órdenes coexisten correctamente en la tabla
✓ Item de orden 69fa85e3f18b7f74c38d6a0a validado correctamente
✓ Item de orden 69fa85e3f18b7f74c38d6a0b validado correctamente
🚩 Idempotencia validada: múltiples órdenes con mismos datos de negocio coexisten sin conflicto gracias a IDs únicos
```

Figura 37. Captura de pantalla de la prueba exitosa con Pytest para validar idempotencia

En la siguiente figura se puede observar a nivel detallado las pruebas de manejo de errores forzando bodies inválidos y documentando el mensaje de error recibido.

```
• Manejo de errores: forzar body inválido y documentar respuestas

🚩 Caso: Body completamente vacío
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404

🚩 Caso: Campos requeridos faltantes (sin cliente_id)
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404

🚩 Caso: Cantidad negativa en item
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404

🚩 Caso: Tipo de dato incorrecto (cantidad como string)
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404

🚩 Caso: cliente_id inexistente
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404

🚩 Caso: Items vacío
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404

🚩 Caso: Body no es JSON (texto plano)
  Status code: 404
  Respuesta: {'detail': 'Not Found'}
  ✓ La API rechazó correctamente el body inválido con 404
```

Figura 38. Captura de pantalla de la prueba exitosa con Pytest con respecto al manejo de errores.

En la siguiente figura se observa la prueba de integración parcial que simula que la API responde bien, pero la base de datos falla.


```
• Prueba de integración parcial: API responde bien pero la base de datos falla
✓ La API responde correctamente con 200 OK
✓ Producto obtenido desde la API: 692a992f056ff25d36115aab - Cereal de avena 250 g

🔥 Simulando fallo de base de datos (cerrando conexión)...
✓ Conexión a la base de datos cerrada

🚨 Intentando insertar orden 69fa89643c73652035507451 con DB caída...
✓ Error capturado al intentar insertar con DB caída: ProgrammingError: Cannot operate on a closed database.

📋 Comportamiento esperado en producción:
- La API debería capturar el error de DB y retornar 503, no 200
- La transacción no debería quedar en estado inconsistente
- El error debería quedar registrado en logs
- El cliente debería poder reintentar de forma segura

⚠️ Riesgo identificado: si la API retorna 200 antes de confirmar la escritura en DB, el cliente asume éxito pero la orden se pierde.
• Verificando que la API sigue disponible tras el fallo de DB...
✓ La API sigue disponible – el fallo de DB no derribó el servicio

🏁 Prueba de integración parcial completada y documentada
```

Figura 39. Captura de pantalla de la prueba exitosa con Pytest de la integración parcial ejecutada

Reporte de ejecución de Newman

Toda la colección de pruebas de Postman fue exportada a un archivo JSON, el cual puede ser ejecutado por medio de Newman para pruebas automatizadas. A continuación se presentan las pruebas del uso del Newman.

```
Integración API - Servicios

❏ Clients
└─ TC-11: Listar todos los clientes
    GET http://localhost:8000/api/mongo/clients/ [200 OK, 132.5kB, 471ms]
    ✓ Código de status es 200
    ✓ Data es un array
    ✓ Objetos de data tienen las propiedades correctas
    ✓ Contenido de respuesta tiene los formatos correctos

└─ TC-12: Listar clientes con parámetros de paginación inválido
    GET http://localhost:8000/api/mongo/clients/?skip=0&limit="hola" [200 OK, 132.5kB, 189ms]
    1. Código de status es 400
    2. Body de respuesta contiene el campo detail
    3. El campo detail es un string
    4. Contenido de respuesta indica que el parámetro limit no es válido

└─ TC-13: Listar cliente específico
    GET http://localhost:8000/api/mongo/clients/692a992f056ff25d36115853 [200 OK, 344B, 88ms]
    ✓ Código de status es 200
    ✓ Body de respuesta tiene los campos correctos con sus tipos
    ✓ Body de respuesta contiene todos los campos del contrato
    ✓ Body de respuesta tiene los campos _id, email y creado con los formatos adecuados

└─ TC-14: Listar cliente específico que no existe
    GET http://localhost:8000/api/mongo/clients/id_no_existente [404 Not Found, 178B, 3ms]
    ✓ Código de status es 404
    ✓ Body de respuesta contiene el campo detail de tipo string
    ✓ Respuesta indica que el cliente no existe
```

Figura 40. Captura de pantalla de pruebas de la entidad Clients en Newman

```

□ Products
└ TC-15: Listar todos los productos con paginación
  GET http://localhost:8000/api/mongo/products/?skip=0&limit=10 [200 OK, 1.89kB, 86ms]
  ✓ El código de estado es 200
  ✓ El body contiene las claves requeridas.
  ✓ Los campos principales tienen el formato correcto.
  ✓ El número de elementos en 'data' coincide con 'limit'.
  ✓ Cada objeto en 'data' tiene los campos requeridos.

└ TC-16: Listar todos los productos con paginación inválida
  GET http://localhost:8000/api/mongo/products/?skip=0&limit="cinco" [422 Unprocessable Content, 302B, 3ms]
  ✓ Código de status es 422
  ✓ El body de la respuesta contiene un arreglo 'detail'.
  ✓ El objeto dentro de 'detail' tiene los campos correctos.
  ✓ El tipo de error y el mensaje son correctos.

└ TC-17: Listar producto específico
  GET http://localhost:8000/api/mongo/products/692a992f056ff25d36115aab [200 OK, 301B, 86ms]
  ✓ El código de estado es 200
  ✓ El objeto contiene las claves requeridas.
  ✓ Los campos principales tienen el formato correcto.
  ✓ Equivalencias contiene al menos codigo_alt y opcionalmente sku

└ TC-18: Listar producto específico con id no existente
  GET http://localhost:8000/api/mongo/products/id_invalido [404 Not Found, 174B, 2ms]
  ✓ El código de estado es 404
  ✓ La respuesta contiene el campo 'detail' como string.
  ✓ El mensaje de error indica que el producto no existe.

```

Figura 41. Captura de pantalla de pruebas de la entidad Products en Newman

```

□ Orders
└ TC-01: Listar todas las órdenes con paginación
  GET http://localhost:8000/api/mongo/order/?skip=0&limit=10 [200 OK, 4.2kB, 192ms]
  ✓ Código de status es 200
  ✓ Body de respuesta tiene los campos total, skip, limit y data.
  ✓ Coherencia de valores, total y precio_unit son mayores a 0 y el número de items es igual al límite.
  ✓ Los datos de los objetos del campo Data e Items tienen el formato correcto.

└ TC-02: Listar todas las órdenes con paginación inválida
  GET http://localhost:8000/api/mongo/order/?skip=0&limit="10" [422 Unprocessable Content, 299B, 3ms]
  ✓ código de status es 422
  ✓ El body de la respuesta contiene un arreglo 'detail'.
  ✓ El objeto dentro de 'detail' tiene los campos correctos.
  ✓ El tipo de error y el mensaje son correctos.

└ TC-04: Listar orden específica con id no existente
  GET http://localhost:8000/api/mongo/order/id_no_existente [404 Not Found, 176B, 3ms]
  ✓ El código de estado es 404
  ✓ La respuesta contiene el campo 'detail' como string.
  ✓ El mensaje de error indica que la orden no existe.

└ TC-05: Crear una orden
  POST http://localhost:8000/api/mongo/order/ [201 Created, 169B, 90ms]
  ✓ El código de estado es 201
  ✓ La respuesta contiene el campo 'order_id'.
  ✓ El campo 'order_id' tiene formato ObjectId.

└ TC-03: Listar orden específica
  GET http://localhost:8000/api/mongo/order/69fa5c4be0e2fe62ba543f4b [200 OK, 393B, 86ms]
  ✓ El código de estado es 200
  ✓ El objeto de respuesta contiene los campos requeridos.
  ✓ Los campos tienen el formato correcto.
  ✓ Los valores son coherentes.

```

Figura 42. Captura de pantalla de primeras pruebas de la entidad Orders en Newman

```

L TC-06: Crear una orden con entradas inválidas
POST http://localhost:8000/api/mongo/order/ [422 Unprocessable Content, 192B, 3ms]
✓ El código de estado es 422
✓ El body de la respuesta contiene el campo 'detail'.
✓ El campo 'detail' es un string y el mensaje de error indica que el cliente_id/producto_id debe ser un ObjectId.

L TC-07: Actualizar una orden
PUT http://localhost:8000/api/mongo/order/69fa5c4be0e2fe62ba543f4b [200 OK, 152B, 89ms]
✓ Código de status es 200
✓ Body de respuesta contiene el campo message de tipo string
✓ Body de respuesta tiene campo mensaje y su valor es order updated

L TC-08: Actualizar una orden con datos incorrectos
PUT http://localhost:8000/api/mongo/order/69fa5c4be0e2fe62ba543f4b [422 Unprocessable Content, 289B, 2ms]
✓ Código de status es 422
✓ Body de respuesta contiene el campo detail de tipo string
✓ Body de respuesta tiene campo detail y su valor indica cuál campo está incorrecto

L TC-09: Eliminar una orden
DELETE http://localhost:8000/api/mongo/order/69fa5c4be0e2fe62ba543f4b [200 OK, 152B, 89ms]
✓ Status code is 200
✓ Body de respuesta contiene el campo message de tipo string
✓ Body de respuesta tiene campo mensaje y su valor es order deleted

L TC-10: Eliminar orden con ID de formato inválido
DELETE http://localhost:8000/api/mongo/order/invalido123 [404 Not Found, 172B, 2ms]
✓ Código de status es 404
✓ Body de respuesta contiene el campo detail de tipo string
✓ Body de respuesta tiene campo detail y su valor indica que el id no existe

```

Figura 43. Captura de pantalla de las últimas pruebas de la entidad Orders en Newman

	executed	failed
iterations	1	0
requests	18	0
test-scripts	18	0
prerequisite-scripts	0	0
assertions	64	4
total run duration: 3s		
total data received: 272.01kB (approx)		
average response time: 82ms [min: 2ms, max: 471ms, s.d.: 111ms]		

#	failure	detail
1.	AssertionError	Código de status es 400 expected response to have status code 400 but got 200 at assertion:0 in test-script inside "Clients / TC-12: Listar clientes con parámetros de paginación inválido"
2.	AssertionError	Body de respuesta contiene el campo detail expected { total: 600, skip: +0, ...(2) } to have property 'detail' at assertion:1 in test-script inside "Clients / TC-12: Listar clientes con parámetros de paginación inválido"
3.	AssertionError	El campo detail es un string expected undefined to be a string at assertion:2 in test-script inside "Clients / TC-12: Listar clientes con parámetros de paginación inválido"
4.	AssertionError	Contenido de respuesta indica que el parámetro limit no es válido expected undefined to deeply equal 'limit parameter not valid' at assertion:3 in test-script inside "Clients / TC-12: Listar clientes con parámetros de paginación inválido"

Figura 44. Captura de resumen de resultados de Newman

Reporte final de pruebas de integración

Esta sección presenta el reporte final del proceso de pruebas de integración. Se incluyen los resultados obtenidos, organizados en una tabla de métricas, seguida de los hallazgos y defectos identificados durante la ejecución.

Tabla de métricas

A continuación se detallan las métricas definidas para este laboratorio junto con los resultados obtenidos al ejecutar la suite de pruebas.

Métrica	Cálculo	Criterio sugerido	Resultado obtenido
Porcentaje de casos aprobados	Casos aprobados / total de casos x 100	>= 85%	$\frac{17}{18 \times 100} = 94.44$
Tiempo promedio de respuesta	Promedio de ms por endpoint crítico	< 1000 ms	74 ms
Defectos encontrados	Cantidad de errores funcionales, técnicos o de contrato	Registrar severidad	4 (severidad baja)
Cobertura de endpoints	Endpoints probados / endpoints del flujo x 100	>= 80% del flujo elegido	100
Validaciones por endpoint	Cantidad de assertions por request	Mínimo 3 por endpoint importante	3 o 4 dependiendo del endpoint
Tasa de error	Respuestas 4xx/5xx inesperadas / total de ejecuciones	Lo más baja posible	$\frac{12}{180} = 0.066$

Hallazgos y defectos

Con respecto a los hallazgos observados durante la ejecución de las pruebas, se puede destacar que el servidor se mantuvo disponible incluso ante una caída de la base de datos, lo cual indica que el servicio tiene cierta resiliencia ante fallos internos, aunque sin retornar un código de error explícito al cliente.

En cuanto al rendimiento, los tiempos de respuesta fueron notablemente bajos, especialmente en los endpoints GET que retornan colecciones completas. Esto se puede atribuir al uso de paginación implementado en la mayoría de estos endpoints, lo cual limita la cantidad de datos retornados por request y mejora considerablemente los tiempos.

La excepción fue GET /clients/, que no contaba con paginación correctamente implementada. Este fue el único caso de prueba que resultó fallido, precisamente por esta razón.

Comparación técnica: Postman vs Pytest

Postman tiene como principal ventaja su facilidad y rapidez de implementación, ya que permite crear pruebas directamente sobre cada endpoint desde su interfaz sin necesidad de escribir código adicional. Además, cuenta con funcionalidades integradas como el Collection Runner, variables de entorno y assertions predefinidos que agilizan la creación de distintos tipos de pruebas. Su limitación principal es la poca flexibilidad en la personalización, tanto en la lógica de las pruebas como en el formato de los resultados, y su estructura tiende a ser rígida por diseño.

Pytest, en cambio, ofrece mayor control sobre cada aspecto de la prueba, desde la lógica de validación hasta el formato de salida. Esto lo hace más adecuado para escenarios complejos o casos que requieren manipulación de datos, como las pruebas contra SQLite en memoria realizadas en este laboratorio. La desventaja es que su implementación toma más tiempo y, al no estar los tests ligados directamente a los endpoints, es más fácil que algún caso quede sin cubrir si no se lleva un registro ordenado de la cobertura.

En resumen, Postman es más conveniente para pruebas rápidas y exploración de endpoints, mientras que Pytest es más apropiado cuando se necesita mayor profundidad y control sobre las pruebas.

Conclusiones

¿Qué diferencia observó entre una prueba de API aislada y una prueba de integración?

En una prueba API aislada, los errores que se pueden encontrar están directamente relacionados con las variables inmediatas que gobiernan el funcionamiento del endpoint. Es decir, se trata de verificar que el contrato básico se cumpla: que el status code sea el esperado, que los parámetros enviados tengan el tipo correcto, que los campos obligatorios estén presentes y que los valores devueltos respeten el formato definido. En este nivel, los fallos suelen ser muy puntuales y fáciles de rastrear: un string donde debería ir un número, un campo faltante en el JSON, un error de validación en la entrada, o un timeout en la llamada.

En cambio, en una prueba de integración, la perspectiva se amplía porque se ponen en juego varias piezas del sistema al mismo tiempo: la API, la base de datos, la lógica de negocio, los servicios externos y las dependencias que los conectan. Aquí los errores pueden ser más sutiles y no siempre aparecen en el punto exacto donde se originan. Un fallo en una etapa inicial puede propagarse y manifestarse más adelante, contaminando el flujo completo. Por ejemplo, si al registrar un producto se guarda mal el identificador, ese error se arrastra cuando se intenta crear una orden, se refleja en los orden_items y finalmente afecta el cálculo del total.

La diferencia esencial está en el alcance del análisis. Una prueba aislada se centra en verificar variables o respuestas inmediatas, buscando fallos puntuales y claros: un campo faltante, un tipo de dato incorrecto, un código de estado inesperado. En una prueba de integración, en cambio, se valida el sistema completo y cómo las partes interactúan entre sí. Aquí los errores pueden propagarse: un dato mal insertado en la base afecta la lógica de negocio y termina reflejándose en cálculos posteriores. El objetivo es comprobar la coherencia global, que todo funcione en conjunto.

¿Qué defectos podrían pasar desapercibidos si solo se prueba la interfaz gráfica?

Si solo se prueba la interfaz gráfica, hay varios defectos que pueden pasar desapercibidos porque no se manifiestan en la capa visual:

- **Errores lógicos en procesos internos:** cálculos incorrectos, validaciones de negocio mal implementadas o reglas que no se cumplen aunque la pantalla muestre datos aparentemente correctos.
- **Problemas en la base de datos:** inserciones duplicadas, relaciones mal guardadas, inconsistencias en claves foráneas o pérdidas de información que no se reflejan de inmediato en la UI.
- **Fallos en integraciones externas:** errores en llamadas a APIs, servicios de terceros o módulos internos que no afectan la presentación pero sí el resultado final.
- **Manejo incorrecto de excepciones:** procesos que fallan silenciosamente y no muestran error en la interfaz, pero dejan el sistema en un estado inconsistente.
- **Problemas de rendimiento o escalabilidad:** tiempos de respuesta lentos, bloqueos bajo carga o consumo excesivo de recursos que no se perciben en una prueba superficial de la UI.

En resumen, probar únicamente la interfaz gráfica permite detectar defectos de presentación (botones, estilos, navegación), pero no garantiza que la lógica de negocio, la persistencia de datos ni las integraciones funcionen correctamente.

¿Qué validaciones fueron más importantes: estado HTTP, contrato JSON, reglas de negocio o persistencia? Justifique.

La validación más importante es el estado HTTP, ya que representa la respuesta definitiva que el cliente recibe del servidor. Un código de estado incorrecto puede hacer que el cliente interprete un fallo como éxito o viceversa, independientemente de lo que contenga el cuerpo de la respuesta. Por ejemplo, si un endpoint retorna 200 cuando debería retornar 503, el cliente asumirá que la operación fue exitosa aunque internamente haya fallado.

Esto no quiere decir que las demás validaciones sean secundarias. El contrato JSON garantiza que los datos lleguen en el formato esperado y evita errores de integración entre servicios. La persistencia asegura que lo que se procesa realmente queda guardado. Y las reglas de negocio son quizás las más críticas desde el punto de vista organizacional, ya que

su incumplimiento puede derivar en inconsistencias de datos, pérdidas económicas o incluso implicaciones legales para la empresa.

En resumen, el estado HTTP es la primera línea de validación, pero las demás capas son igualmente necesarias para garantizar un sistema íntegro y confiable.

¿Qué riesgos tendría integrar un proveedor externo de pagos sin pruebas automatizadas?

Integrar un proveedor externo de pagos sin pruebas automatizadas expone el sistema a riesgos financieros y de seguridad difíciles de detectar manualmente. El caso más crítico es el de cobros duplicados: si no existe una prueba que valide que el sistema rechaza un segundo intento de pago para la misma transacción, un error en el flujo podría resultar en cargos dobles al usuario sin que el equipo lo detecte hasta que lleguen los reclamos.

Otro riesgo importante es el de regresiones. Cada vez que se agrega un nuevo módulo o se modifica uno existente, existe la posibilidad de romper algo que antes funcionaba correctamente. Sin pruebas automatizadas, esa verificación depende enteramente del criterio del desarrollador, lo que deja margen para que vulnerabilidades o comportamientos incorrectos pasen desapercibidos antes de llegar a producción.

¿Qué mejoraría en su estrategia de pruebas si este proyecto fuera para producción?

Si este proyecto fuera para producción, el primer cambio sería integrar las pruebas en un pipeline de CI/CD mediante herramientas como GitHub Actions o Jenkins, de forma que se ejecuten automáticamente con cada pull request. Complementario a esto, se configuraría una regla de protección en la rama principal para que ningún cambio pueda ser integrado si las pruebas no pasan, eliminando así la dependencia del criterio individual del desarrollador.

En cuanto a la cobertura, ampliaría la cantidad de casos de prueba para cubrir más escenarios límite y flujos alternativos, especialmente en los endpoints de creación y actualización de órdenes. También se agregarían pruebas de carga para validar el comportamiento del sistema bajo volúmenes altos de requests, algo que en un entorno de producción real es indispensable.

Finalmente, se separarían los ambientes de prueba y producción de forma más estricta, asegurando que las pruebas nunca interactúen con datos reales y que el mock server cubra todos los casos de fallo posible del proveedor externo.

Referencias bibliográficas

MDN Web Docs. (2025). *HTTP response status codes*. Mozilla.
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>